The logical theory of temporal graphs

Internship report M1

Igor Semezies,
under the supervision of Martin Lange

February-July 2026
University of Kassel, Germany

> The creatures were friendly, and they could see in four dimensions. They pitied Earthlings for being able to see only three. They had many wonderful things to teach Earthlings, especially about time. Billy promised to tell what some of those wonderful things were in his next letter.
>
> *Slaughterhouse-Five*, Kurt Vonnegut

Abstract

A *temporal graph* is a graph whose edge relation evolves (that is, where edges may appear or disappear) along a discrete time. Such a structure combines two dimensions: a dimension of *space* (provided by the graph structure) and a dimension of *time*. This document investigates the model-theoretic and computational properties of various modal logics interpreted on temporal graphs. Our main result concerns a μ-calculus-ish logic, $\mathtt{TG}\mu$ – whose introduction is motivated by its potential links with temporal graph neural networks –, for which we prove a doubly-exponential small model property (i.e. every satisfiable formula $\varphi \in \mathtt{TG}\mu$ has a model of size $\lesssim 2^{2^{|\varphi|}}$), and give an ExpSpace algorithm for the satisfiability problem. The principal ingredient of our proof is the introduction of a new kind of automata, that take temporal graphs as inputs. We also introduce and rely on the use of a generalisation of temporal graphs, that we call *quasi temporal graphs*. We argue that quasi temporal graphs actually form a more natural class of models for our spatio-temporal logics than temporal graphs. Finally, we show that the doubly-exponential upper bound for the small model property of $\mathtt{TG}\mu$ is optimal, by providing matching lower bounds: we define a sequence $(\Phi_n)_{n\geq 0}$ of satisfiable formulae of $\mathtt{TG}\mu$, such that $|\Phi_n| \simeq n$ and whose smallest models have size $\simeq 2^{2^n}$.

Acknowledgements

I would first like to thank my supervisor Martin. Being under his wing for these past few months has taught me a lot of things, not only on scientific aspects – it is a platitude to say that my understanding of *temporal logics in computer science*, in particular μ-calculi, has deepened immeasurably since the beginning of my internship –, but also on what actually is the job of a researcher, thanks to all the opportunities of activities he gave me (publishing a paper, reviewing a paper, running a tutorial...). PSG knows well the taste of success now, so I wish the best to FC Bayern for the coming years.

I also thank the whole TI/FM group and its closure for their caring welcome: many thanks to Eric, Florian, Lars-Eric, Marco, Martin, Maurice, Michael, Norbert, Sören and Tina (whom I thank in particular for handling the administrative hell that was my internship agreement at the beginning of my stay).

I have a heartfelt thought for my alpaca companion Ludwig, to whom I wish good hikes and adventures in the Diemel valley.

I am grateful to my teachers Mihaela Sighireanu and Stéphane Demri for their help, support and advice during these past two years at the ENS. I thank Stéphane in particular for helping me getting in contact with Martin.

Finally, I would like to express my gratitude to my parents for having put everything in place so that I only have to worry about my scientific problems.

Contents

Acknowledgements **2**

Introduction **4**

1. Structures **8**
1.1. (Quasi) Temporal Graphs 8
1.2. Spatio-temporal bisimulation 9
1.3. From QTGs to TGs 11

2. Logics **11**
2.1. The logic $\mathtt{LTL}\times\mathtt{K}$ 12
2.2. The temporarily-guarded μ-calculus $\mathtt{TG}\mu$ 14
2.3. Expressiveness comparison 19
2.4. Bisimilarity and elementary equivalence 20
2.5. The philosophy of QTGs 22

3. Spatio-temporal automata **23**
3.1. Definition 23
3.2. The nonemptiness test 25
3.3. Representing a spatio-temporal automaton in a computer 28

4. Small model property **29**
4.1. From $\mathtt{TG}\mu$ to automata 29
4.2. Consequences on expressiveness 32
4.3. The satisfiability algorithm 34

5. Matching lower bounds **34**
5.1. Lower bound for QTGs 35
5.2. Lower bound for TGs 38

Conclusion **39**

A. From QTGs to TGs: the proof **44**
A.1. Step 1: building $\mathfrak{J}$ 44
A.2. Step 2: building $\mathfrak{G}$ 46

Introduction

Artificial intelligence, and notably the subfield of machine learning, has enabled major technological advancements in all sorts of industrial and scientific fields. In particular, some applications of machine learning models involve safety-critical systems, where concerns regarding safety, security or explainability arise. In fact, the black-box nature of machine learning is often in direct conflict with such concerns, so this line of research has proven to be particularly challenging.

The fields of verification and formal methods offer a collection of techniques to match these challenges. One of them is to analyse the expressive power of machine learning models, that is, understanding what kinds of functions they can compute. Expressiveness studies are often done by relating new computational devices – in our case, machine learning models – to known formalisms such as logics or automata.

Recent years have seen important progress in understanding the expressiveness of a specific machine learning model: *graph neural networks* (GNNs). GNNs can essentially be abstracted as functions that take a pointed graph[1] as input, and return "true" or "false". Thus, they can directly be compared in terms of expressiveness to logics on graphs, such as (fragments of) first-order logic with one binary predicate for the edge relation, or (extensions of) the basic modal logic $\mathtt{K}$ [BKM$^+$20, BLMT24, NSST24].

As the expressive power of GNNs is now relatively well understood, there have been a growing interest in studying more complex neural network architectures, notably *temporal graph neural networks* (TGNNs). TGNNs [LLS$^+$23] work roughly like GNNs, except that they take pointed *temporal graphs* (instead of pointed graphs) as inputs. A temporal graph (TG) is a finite sequence of graphs that all share the same node set; it is usually interpreted as a single graph whose edge relation evolves (that is, where edges may appear or disappear) along a discrete time. To give a concrete example, [PDC$^+$20] applies TGNNs to various TG-shaped data sets, including one that represents the Reddit hyperlink network[2]: the nodes are subreddits, that is, Reddit communitites, and the edges are hyperlinks from a post in a subreddit to another, marked with the time of the post (hence the temporal dependency) and "the sentiment of the source community post towards the target community post" (the goal is to study toxic interactions between subreddits).

Sälzer et al. [SWL25] suggested to analyse the expressiveness of TGNNs by relating them to specific *spatio-temporal logics*, that is, logics that can specify properties of (pointed) temporal graphs. These logics contain two types of modal operators: temporal operators ($\bigcirc$ and $\ominus$) that are used to navigate through the temporal sequence, and spatial operators ($\Diamond$ and $\Box$) used to traverse through a single graph at a given timestamp. Although the theory of multi-dimensional logics (which encompasses our case of two dimensions, one dimension of time and one of space) has been developed for a few decades and is now well established [GKWZ03], it does not include the specific case

[1] That is, a pair (G, v), where G is a graph and v is a node of G.

[2] `http://snap.stanford.edu/data/soc-RedditHyperlinks.html`

of logics interpreted on temporal graphs (so *a fortiori* not the logics defined in [SWL25]).[3]

The present work aims at filling this gap in the theory of multi-dimensional logics, by studying the model-theoretic and computational properties of some logics interpreted on temporal graphs.

Specifically, we study two logics interpreted on temporal graphs: $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$. $\mathtt{LTL}\times\mathtt{K}$ is obtained by combining the Linear-time Temporal Logic LTL interpreted on finite traces [GV13] (for the temporal dimension) and the standard modal logic K (for the spatial dimension). Since LTL has some (limited) recursive mechanism due to the Until operator and K has no recursive mechanism, formulae of $\mathtt{LTL}\times\mathtt{K}$ can talk about events happening arbitrarily far in the future, but within a single graph at a given timestamp they can only explore nodes to a bounded depth. This corresponds exactly to how TGNNs operate: at a given timestamp, they can only explore the neighbourhood of a node at a radius bounded by their number of layers. In fact, $\mathtt{LTL}\times\mathtt{K}$ is the main logic used in [SWL25] to be compared in terms of expressiveness with TGNNs.

The *temporarily-guarded μ-calculus* $\mathtt{TG}\mu$ is an extension of $\mathtt{LTL}\times\mathtt{K}$ that can essentially be understood as replacing LTL by the linear-time μ-calculus $\mathtt{LT}\mu$ [Var88, BB89, LDTC16] (again, interpreted on finite traces) for the temporal part. Crucially, the so-called "temporal guardedness" ensures that formulae in $\mathtt{TG}\mu$ can only explore nodes of a single graph to a bounded depth, so that $\mathtt{TG}\mu$ is, just as (and maybe even more than) $\mathtt{LTL}\times\mathtt{K}$, a relevant yardstick to measure the expressiveness of TGNNs.

Thus, although our work is only concerned with logics (we do not address any topic related to machine learning), the choice of the objects of study (TGs and the logics $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$) is always guided by the applications to the verification of TGNNs.

Our analysis of $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$ is done by studying two problems: the small model property and the complexity of satisfiability. A small model property for a logic $\mathcal{L}$ consists in finding a (computable) function $f : \mathbb{N}_0 \to \mathbb{N}_0$ such as every satisfiable formula $\varphi \in \mathcal{L}$ has a model $\mathfrak{M}$ such that $|\mathfrak{M}| \leq f(|\varphi|)$. The complexity of satisfiability consists in determining the computational complexity of the decision problem "Input: $\varphi \in \mathcal{L}$; Output: true iff φ is satisfiable". These two problems are closely related. As an example, any small model property f immediately yields an algorithm for the satisfiability problem (provided that model-checking in $\mathcal{L}$ is decidable): given an input formula $\varphi \in \mathcal{L}$, enumerate all the structures $\mathfrak{M}$ of size $\leq f(|\varphi|)$, and check whether $\mathfrak{M} \models \varphi$. More generally, the mathematical structures used to solve one problem can usually (and it will indeed be the case in the present work) be adapted to solve the other.

Contribution. We show that $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$ have a doubly-exponential small model property, i.e. that the above statement holds for the function $f(n) = 2^{2^n}$. We also show that this upper bound can be matched (and is therefore optimal), by defining a sequence of satisfiable formulae $(\Phi_n)_{n\geq 0}$ in $\mathtt{LTL}\times\mathtt{K}$ (which can in particular be seen as $\mathtt{TG}\mu$ formulae, since $\mathtt{LTL}\times\mathtt{K}$ can be embedded into $\mathtt{TG}\mu$), such that $|\Phi_n| \simeq n$ and whose smallest models have size $\simeq 2^{2^n}$. Finally, we give an algorithm in ExpSpace to decide

[3] See the "Related work" paragraph for a more detailed discussion.

satisfiability in $\mathtt{TG}\mu$ (and therefore in $\mathtt{LTL}\times\mathtt{K}$ as well). (We however do not prove that this problem is also ExpSpace-hard, so we do not know if the ExpSpace complexity upper bound is optimal.)

By comparison, the three one-dimensional logics $\mathtt{LTL}$, $\mathtt{LT}\mu$ and $\mathtt{K}$ have a singly-exponential small model property and a PSpace-complete satisfiability problem [SC85, Var88, DGL16], so it seems that combining them into two-dimensional logics interpreted on TGs increase exponentially their small model property as well as their satisfiability complexity.

The needs of the proofs of the aforementionned results lead us to introduce other mathematical objects related to TGs, that we believe to be of independent interest. Specifically, *spatio-temporal automata* – a class of automata that take pointed TGs as inputs, and that we introduce – are central points in the proofs our main results; they can also be seen as computational devices for TGs, enabling us to analyse their expressiveness, notably by comparing it with our spatio-temporal logics.

Also, while we believe that $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$ are good logical languages to reason on TGs, the rigidity of TGs makes them not the best structures to interpret our spatio-temporal logics. When trying to find a model of an arbitrary formula, we will naturally encounter the notion of *quasi temporal graph* (QTG), which is more flexible. QTGs are a generalisation of TGs[4], that relax the necessity of all graphs to share the same node set. Any QTG can effectively be turned into an (essentially) equivalent TG, at the cost of some increasing in the size of the involved graphs. Therefore, at the proof-theoretical level, the classes of QTGs and TGs are equivalent, but QTGs are more concise than TGs: we will actually prove that $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$, when interpreted on the broader class of QTGs, have a slightly better small model property than when interpreted on TGs.

Structure of the document. In Section 1, we present QTGs and TGs, and in Section 2, we define the logics $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$. In Section 3, we introduce spatio-temporal automata, and prove some results on them that will be directly used in Section 4, where we prove the small model property of $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$ and present the satisfiability algorithm. Finally, in Section 5, we provide the matching lower bounds for the small model property.

Related work.

Multi-dimensional logics. Multi-dimensional logics [GKWZ03, MV97] offer a general framework for combining modal logics, each capturing a distinct aspect of reasoning such as time, space, or knowledge. The standard theory distinguishes *fusion* logics, in which the component logics do not interact (apart from the shared atomic propositions), from *product* logics, in which there is a strong interaction between the component logics, produced by requiring the models to have a Cartesian product structure.

[4] Actually, most of the results of the document will be proven on the (larger) class of QTGs, and will in particular hold for TGs, by restriction.

The logics defined in [SWL25] and this document do not fall in any of these categories. Notably, despite the (misleading?) notation, $\texttt{LTL}\times\texttt{K}$ is not a product of LTL and K, at least not according to the standard terminology: product logics in [GKWZ03] are interpreted on what would correspond in our case to temporal graphs with *static edge relations* (so the temporal dependency only lies in the labeling of the nodes), which enforces (what corresponds, in our notations, to) the axioms

$$\Diamond \bigcirc p \leftrightarrow \bigcirc \Diamond p$$
$$\bigcirc \Box p \rightarrow \Box \bigcirc p.$$

These are not axioms anymore when interpreting our logics on arbitrary temporal graphs.

The work that is most closely related to the present study is on temporal description logics [LWZ08]. The structures known as temporal graphs in the machine learning community are known as "temporal interpretations over a constant domain" in the description logic dialect. The logic $\texttt{LTL}\times\texttt{K}$, as considered here, could be seen as a simple description logic over such structures. A minor difference lies in the treatment of time: in the present work, the time line is finite, whereas it is infinite in [LWZ08]. More important differences are that 1. we also study an extension of $\texttt{LTL}\times\texttt{K}$, namely $\texttt{TG}\mu$, which is not done in [LWZ08], 2. the context of description logics makes [LWZ08] focus on different reasoning problems where complexity is introduced via TBox axioms which is not present here, and 3. our QTGs do not seem to correspond to their "temporal interpretations over a varying domain".

The TIME paper. The author's stay in Kassel led to a publication at the conference *TIME 2026* (the paper is [ALS26]), which will be held in Cork in September 2026. There is actually not much in common between the TIME paper and the present work. Indeed, while both documents claim to interpret $\texttt{LTL}\times\texttt{K}$ and $\texttt{TG}\mu$ on QTGs, their definitions of QTGs are different: the QTGs of the TIME paper (that we may call *asynchronous QTGs*) allow the time lines of different nodes to end at different timestamps, whereas this is not permitted by the QTGs of the present work (that we may call *synchronous QTGs*), *a fortiori* not by TGs. The class of asynchronous QTGs is larger than the class synchronous QTGs, *and not equivalent at the proof-theoretical level to the class of TGs anymore*: the formula $\bigcirc\mathsf{tt} \wedge \Diamond \ominus \mathsf{ff}$ is satisfiable in an asynchronous QTG, but not in a TG (and not in a synchronous QTG either). Interpreting $\texttt{LTL}\times\texttt{K}$ and $\texttt{TG}\mu$ on asynchronous QTGs actually leads to a much smaller complexity for the satisfiability problems, namely PSpace-complete for $\texttt{LTL}\times\texttt{K}$ and ExpTime-complete for $\texttt{LTL}\times\texttt{K}$ and $\texttt{TG}\mu$ respectively (whereas our best algorithm to decide satisfiability for these logics, when interpreted on synchronous QTGs and TGs, is in ExpSpace).

The only parts that are common to the TIME paper and the present work are 1. some paragraphs in their introductions, 2. the syntax of $\texttt{LTL}\times\texttt{K}$ and $\texttt{TG}\mu$ and 3. the Hennessy-Milner property (Proposition 2.13 here).

Notations. In all the document, we fix a nonempty set AP of atomic propositions, and a nonempty set Var of (monadic second-order) variables.

Given a set X, we write $\mathcal{P}(X)$ for the powerset of X, that is, the set of all subsets of X. We write $\mathbb{N}_0 = \{0, 1, 2, \ldots\}$ for the set of natural integers, and $\mathbb{N}_1 = \mathbb{N}_0 \setminus \{0\}$. We write $\mathbb{Z}$ for the set of (all) integers. Given $a, b \in \mathbb{Z}$, we note $[\![a, b]\!] = \{a, a+1, \ldots, b-1, b\}$ (with the convention that $[\![a, b]\!] = \varnothing$ whenever $a > b$). For $n \in \mathbb{N}_0$, we note $[n] = [\![0, n-1]\!]$.

We define a *graph* as a triple $G = (V, E, L)$, where V is a finite set of *nodes*, $E \subset V \times V$ is the *edge relation*, and $L : V \to \mathcal{P}(\mathsf{AP})$ is the *labeling* of the nodes.

1. Structures

This section introduces the structures on which our spatio-temporal logics will be interpreted: temporal graphs (TGs) and quasi temporal graphs (QTGs). In Section 1.1, we define these two structures. In Section 1.2, we extend bisimulation and bisimilarity to the spatio-temporal setting, thus equipping the class of (Q)TGs with a notion of behavioural equivalence. We use this notion of equivalence in Section 1.3 to formalise the fact that any QTG can be turned into a TG that contains "at least as much information".

1.1. (Quasi) Temporal Graphs

Since QTGs are meant to generalise TGs, we first introduce the class of QTGs, and we will then see TGs as a special case of QTGs.

A QTG is a sequence of graphs G_i with set of nodes V_i (where $i = 0, \ldots, n-1$ represents a discrete time) that are connected by temporal successor functions $\sigma_i : V_i \to V_{i+1}$. Given a node $v \in V_i$ at timestamp i, $\sigma_i(v)$ represents what v will become at the next timestamp $i + 1$.

Definition 1.1 (Quasi temporal graph)

A *quasi temporal graph* (QTG) is a tuple $\mathfrak{L} = (n, (G_i)_{i\in[n]}, (\sigma_i)_{i\in[n-1]})$, where:

- $n \in \mathbb{N}_1$ is the *length* of $\mathfrak{L}$;
- For all $i \in [n]$, $G_i = (V_i, E_i, L_i)$ is a graph;
- For all $i \in [n-1]$, $\sigma_i : V_i \to V_{i+1}$ is the i-th *temporal successor function*.

We now introduce some useful notation: for $0 \leq i \leq j < n$, we write

$$\sigma_{ij} = \sigma_{j-1} \circ \cdots \circ \sigma_i : V_i \to V_j.$$

In the above interpretation, given some node $v \in V_i$ at timestamp i, $\sigma_{ij}(v)$ represents what v will become in the future, at timestamp j. As an example, $\sigma_{ii} = \mathrm{id}_{V_i}$, and $\sigma_{i,i+1} = \sigma_i$.

For $i \in [n]$ and $v \in V_i$, we write

$$\mathrm{Nei}_i(v) = \{w \in V_i \mid (v, w) \in E_i\}$$

for the set of (spatial) neighbours of v at timestamp i.

To measure the size of a QTG, we use two metrics (one for each dimension): the *length* $\mathrm{len}(\mathfrak{L}) = n$, and the *surface* $\mathrm{surf}(\mathfrak{L}) = \max_{i \in [n]} |V_i|$.

Finally, we write

$$\mathrm{Pts}(\mathfrak{L}) = \{(v, i),\ i \in [n],\ v \in S_i\}$$

for the set of *points* of $\mathfrak{L}$. A *pointed QTG* is a triple $(\mathfrak{L}, v, i)$, where $\mathfrak{L}$ is a QTG and $(v, i) \in \mathrm{Pts}(\mathfrak{L})$. Formulae of our spatio-temporal logics will be interpreted on pointed QTGs.

> **Remark 1.2**
> The asynchronous QTGs of the TIME paper (see the Introduction section) are defined as these QTGs, except that the temporal successor functions σ_i are partial and not total.

A TG is a QTG in which the identity of a node does not change over time; that is, the set of nodes $V_i = V$ is static, and for every node $v \in V$, $\sigma_i(v) = v$ (i.e. the temporal successor functions are identity functions). It can equivalently be seen as a single graph in which the edge relation and labeling of the nodes evolve along a discrete time i.

Definition 1.3 (Temporal graph)

A *temporal graph* (TG) is a QTG $\mathfrak{G}$ such that:

- $V_0^{\mathfrak{G}} = \cdots = V_{n-1}^{\mathfrak{G}} = V$;
- $\sigma_0^{\mathfrak{G}} = \cdots = \sigma_{n-2}^{\mathfrak{G}} = \mathrm{id}_V$.

There are two main differences between a general QTG and a TG. First, in a QTG, the temporal successor functions σ_i need not be injective: it is possible that two different nodes $v \neq w \in V_i$ at timestamp i are merged together at timestamp $i + 1$, that is, $\sigma_i(v) = \sigma_i(w)$ (whereas in a TG, two distinct nodes will always stay distinct in the future). Second, in a QTG, the temporal successor functions no need to be surjective: it is possible that a node has no "past" (whereas in a TG, a node v at some timestamp $i > 0$ will always exist at timestamp $i - 1$). A QTG in which the temporal successor functions are bijective is in fact a TG, up to some renaming of the nodes.

1.2. Spatio-temporal bisimulation

We introduce spatio-temporal bisimulation, which is the notion of behavioural equivalence that suits to (Q)TGs. It is a straightforward extension of the well-knowm bisimulation on graphs: we require that any *spatial or temporal* step in one structure can be simulated in the other, and vice-versa. As such, the basic properties of the standard bisimulation on graphs all smoothly generalise to the spatio-temporal setting, as we show below.

We will see in Section 2.4 that the induced notion of bisimilarity characterises elementary equivance for our spatio-temporal logics, which supports our definition as "the right way" to generalise bisimulation to spatio-temporal structures.

Definition 1.4 (Spatio-temporal bisimulation)

Let $\mathfrak{A}$, $\mathfrak{B}$ be two QTGs. We say that a relation $\beta \subset \mathrm{Pts}(\mathfrak{A}) \times \mathrm{Pts}(\mathfrak{B})$ is a *(spatio-temporal) bisimulation* between $\mathfrak{A}$ and $\mathfrak{B}$ if, for all $(v,i) \in \mathrm{Pts}(\mathfrak{A})$, $(w,j) \in \mathrm{Pts}(\mathfrak{B})$ such that $(v,i)\beta(w,j)$, we have:

(1) $L_i^{\mathfrak{A}}(v) = L_j^{\mathfrak{B}}(w)$;

(2) $\forall v' \in \mathrm{Nei}_i^{\mathfrak{A}}(v),\ \exists w' \in \mathrm{Nei}_j^{\mathfrak{B}}(w),\ (v',i)\beta(w',j)$;

(3) $\forall w' \in \mathrm{Nei}_j^{\mathfrak{B}}(w),\ \exists v' \in \mathrm{Nei}_i^{\mathfrak{A}}(v),\ (v',i)\beta(w',j)$;

(4) $i < \mathrm{len}(\mathfrak{A}) - 1$ iff $j < \mathrm{len}(\mathfrak{B}) - 1$, and in case they do,

$$(\sigma_i^{\mathfrak{A}}(v), i+1)\beta(\sigma_j^{\mathfrak{B}}(w), j+1).$$

In the rest of the document, we will not deal with the standard bisimulation on graphs, so we will simply say "bisimulation" for a spatio-temporal bisimulation.

It is easily checked that an arbitrary union of bisimulations between two QTGs $\mathfrak{A}$ and $\mathfrak{B}$ is still a bisimulation. In particular, the union of all bisimulations between $\mathfrak{A}$ and $\mathfrak{B}$ is itself a bisimulation: we call it the *bisimilarity relation* between $\mathfrak{A}$ and $\mathfrak{B}$.

Definition 1.5 (Bisimilarity)

For any two QTGs $\mathfrak{A}$, $\mathfrak{B}$, we write $\mathrm{B}_{\mathfrak{A},\mathfrak{B}}$ for the union of all bisimulations between $\mathfrak{A}$ and $\mathfrak{B}$. It is the biggest bisimulation between $\mathfrak{A}$ and $\mathfrak{B}$.

Let $\rightleftarrows$ be the binary relation on the class of pointed QTGs defined by

$$(\mathfrak{A}, v, i) \rightleftarrows (\mathfrak{B}, w, j) \quad \text{iff} \quad (v,i)\mathrm{B}_{\mathfrak{A},\mathfrak{B}}(w,j).$$

The relation $\rightleftarrows$ is called *bisimilarity*. If $(\mathfrak{A}, v, i) \rightleftarrows (\mathfrak{B}, w, j)$, we say that $(\mathfrak{A}, v, i)$ and $(\mathfrak{B}, w, j)$ are *bisimilar*.

By definition, $(\mathfrak{A}, v, i) \rightleftarrows (\mathfrak{B}, w, j)$ iff there exists a bisimulation β between $\mathfrak{A}$ and $\mathfrak{B}$ such that $(v,i)\beta(w,j)$.

It is a well-known fact [DGL16] that the standard one-dimensional bisimilarity is an equivalence relation on the class of pointed graphs. This stays true in the spatio-temporal setting, with exactly the same proof.

Proposition 1.6

The relation $\rightleftarrows$ is an equivalence relation on the class of pointed QTGs.

1.3. From QTGs to TGs

For two QTGs $\mathfrak{A}$, $\mathfrak{B}$, we write $\mathfrak{A} \preceq \mathfrak{B}$ if:

$$\forall p \in \mathrm{Pts}(\mathfrak{A}),\ \exists q \in \mathrm{Pts}(\mathfrak{B}),\ (\mathfrak{A}, p) \rightleftarrows (\mathfrak{B}, q).$$

$\mathfrak{A} \preceq \mathfrak{B}$ means that $\mathfrak{B}$ contains at least as much information as $\mathfrak{A}$, in the sense that for any landscape that can be admired at a place p of $\mathfrak{A}$, we can find an akin one at some place q of $\mathfrak{B}$. The relation $\preceq$ is a preorder on the class of QTGs.

The following theorem says that any QTG can be turned into a TG that contains at least as much information, and whose (temporal and spatial) size can be controlled.

Theorem 1.7

Let $\mathfrak{L}$ be a QTG. Then there exists a TG $\mathfrak{G}$ such that:

- $\mathfrak{L} \preceq \mathfrak{G}$;
- $\mathrm{len}(\mathfrak{G}) = \mathrm{len}(\mathfrak{L})$, and $\mathrm{surf}(\mathfrak{G}) \leq \mathrm{len}(\mathfrak{L}) \cdot \mathrm{surf}(\mathfrak{L})$.

Proof sketch. While the construction is easy to depict visually, writing it formally is quite tedious. Hence, we only give the main ideas here, and let the formal construction to Appendix A.

We proceed in two steps. First, we build a QTG $\mathfrak{J}$ such that $\mathfrak{L} \preceq \mathfrak{J}$ and all the temporal successor functions of $\mathfrak{J}$ are injective. To do that, for $k \in [n]$, whenever we have distinct nodes $v_1, \dots, v_s \in V_k^{\mathfrak{L}}$ such that $\sigma_k^{\mathfrak{L}}(v_1) = \cdots \sigma_k^{\mathfrak{L}}(v_s) = w$, we duplicate w to have s copies $w_1, \dots, w_s$, and let, for all $j \in [\![1, s]\!]$, $\sigma_k^{\mathfrak{J}}(v_j) = w_j$. We thereby make the k-th temporal successor function of $\mathfrak{J}$ injective. To ensure that we do not break the work we did for the timestamp k when dealing with another timestamp k', we proceed by increasing k, that is, we first deal with $k = 0$, then $k = 1$, etc. until $k = n - 2$.

Second, we modify $\mathfrak{J}$ to make its temporal successor functions surjective, resulting in a TG $\mathfrak{G}$ such that $\mathfrak{J} \preceq \mathfrak{G}$. To do that, for every $i \in [n]$ and $v \in V_i^{\mathfrak{J}} \setminus \mathrm{Im}(\sigma_{i-1}^{\mathfrak{J}})$, we add temporal predecessors of v at timestamps $0, \dots, i-1$. ◀

2. Logics

In this section, we present the logics that we shall spend the rest of the document studying. These are modal logics interpreted on QTGs.

We start the section by defining the two logics we are interested in: LTL×K (Section 2.1) and the *temporarily-guarded* μ-calculus TGμ (Section 2.2). We then compare their expressiveness (Section 2.3) and show that they all satisfy the Hennessy-Milner property w.r.t. spatio-temporal bisimulation (Section 2.4). (The results of these two sections are straightforward extensions to the spatio-temporal setting of results which are standard

in the one-dimensional setting, see e.g. [DGL16].) We end the section by an informal paragraph to explain why we believe that QTGs form a more natural class of models for our logics than TGs (Section 2.5).

2.1. The logic LTL×K

The syntax of LTL×K is built from propositional logic by adding the modal operators of LTL (the *next* $\bigcirc$, the *weak next*[5] $\ominus$, the *until* U and the *release* R) and K (the *diamond* $\Diamond$ and the *box* $\Box$) together.

Definition 2.1 (Syntax of LTL×K)

The syntax of LTL×K is inductively defined as follows:

$$\begin{aligned}\varphi ::= {} & \mid p \mid \neg p && (p \in \mathsf{AP}) \\ & \mid \varphi \vee \varphi \mid \varphi \wedge \varphi \\ & \mid \bigcirc\varphi \mid \ominus\varphi \mid \varphi \,\mathtt{U}\, \varphi \mid \varphi \,\mathtt{R}\, \varphi \\ & \mid \Diamond\varphi \mid \Box\varphi\end{aligned}$$

We write LTL×K for the set of formulae generated by the above grammar.

We use the standard abbreviations $\mathtt{F}\varphi = \mathsf{tt}\,\mathtt{U}\,\varphi$ and $\mathtt{G}\varphi = \mathsf{ff}\,\mathtt{R}\,\varphi$. We write BTL×K for the fragment of LTL×K that does not have the operators U and R (BTL stands for *Basic Temporal Logic*, that is, the linear-time logic that only contains the modal operators $\bigcirc$ and $\ominus$).

The temporal operators of LTL enable us to describe the evolution of a node with time (using the temporal successor functions σ_i), and the spatial operators of K enable us to traverse through a single graph at a fixed timestamp. As an example, the formula

$$\varphi = p \wedge \Diamond(q \vee (\mathtt{F}r)) \in \mathtt{LTL}\times\mathtt{K}$$

is satisfied at a point (v, i) of a (Q)TG $\mathfrak{L}$ iff (v, i) satisfies p and has a neighbour (in the i-th graph $G_i^{\mathfrak{L}}$) w such that either (w, i) satisfies q or there exists a timestamp $j \geq i$ such that $(\sigma_{ij}(w), j)$ satisfies r (as a reminder, $\sigma_{ij}(w)$ is only what w becomes in the future, at timestamp j).

Formally, the semantics of LTL×K are defined as follows.

Definition 2.2 (Semantics of LTL×K)

Let $\mathfrak{L}$ be a QTG. We define the satisfaction relation $\models$ inductively on the formula $\varphi \in \mathtt{LTL}\times\mathtt{K}$, as follows:

[5] The difference between the (strong) next ($\bigcirc$) and the weak next ($\ominus$) is that $\bigcirc$ requires that tomorrow exists and $\ominus$ does not. Hence, $\bigcirc\varphi$ will be false at the last timestamp of a linear-time sequence, while $\ominus\varphi$ will be true (see Definition 2.2). The operator $\ominus$ is the dual of $\bigcirc$. When LTL is interpreted on infinite sequences, there is no difference between $\bigcirc$ and $\ominus$ ($\bigcirc$ is self-dual).

For all $(v, i) \in \mathrm{Pts}(\mathfrak{L})$,

$$
\begin{array}{lll}
\mathfrak{L}, v, i \models p & \text{iff} & p \in L_i^{\mathfrak{L}}(v) \\
\mathfrak{L}, v, i \models \neg p & \text{iff} & p \notin L_i^{\mathfrak{L}}(v) \\
\mathfrak{L}, v, i \models \varphi \vee \psi & \text{iff} & \mathfrak{L}, v, i \models \varphi \ \text{ or } \ \mathfrak{L}, v, i \models \psi \\
\mathfrak{L}, v, i \models \varphi \wedge \psi & \text{iff} & \mathfrak{L}, v, i \models \varphi \ \text{ and } \ \mathfrak{L}, v, i \models \psi \\
\mathfrak{L}, v, i \models \bigcirc \varphi & \text{iff} & i < \mathrm{len}(\mathfrak{L}) - 1 \ \text{ and } \ \mathfrak{L}, \sigma_i(v), i+1 \models \varphi \\
\mathfrak{L}, v, i \models \ominus \varphi & \text{iff} & i = \mathrm{len}(\mathfrak{L}) - 1 \text{ or } (i < \mathrm{len}(\mathfrak{L}) - 1 \text{ and } \sigma_i(v), i+1 \models \varphi) \\
\mathfrak{L}, v, i \models \varphi \,\mathtt{U}\, \psi & \text{iff} & \exists k \in [\![i, \mathrm{len}(\mathfrak{L}) - 1]\!], \ \begin{cases} \mathfrak{L}, \sigma_{ik}(v), k \models \psi \\ \forall j \in [\![i, k-1]\!], \ \mathfrak{L}, \sigma_{ij}(v), j \models \varphi \end{cases} \\
\mathfrak{L}, v, i \models \varphi \,\mathtt{R}\, \psi & \text{iff} & \text{either } \mathfrak{L}, v, i \models \psi \,\mathtt{U}(\varphi \wedge \psi) \text{ or } \forall j \in [\![i, \mathrm{len}(\mathfrak{L}) - 1]\!], \ \mathfrak{L}, v, i \models \psi \\
\mathfrak{L}, v, i \models \Diamond \varphi & \text{iff} & \exists w \in \mathrm{Nei}_i^{\mathfrak{L}}(v), \ \mathfrak{L}, w, i \models \varphi \\
\mathfrak{L}, v, i \models \Box \varphi & \text{iff} & \forall w \in \mathrm{Nei}_i^{\mathfrak{L}}(v), \ \mathfrak{L}, w, i \models \varphi.
\end{array}
$$

Example 2.3 (A formula that is satisfied by fat TGs and thin QTGs) We introduce an $\mathtt{LTL}\times\mathtt{K}$ formula that we will use later on in the document (Section 5.2). Let

$$\sigma = \mathtt{G}\left[\neg p \wedge \Diamond(p \wedge \ominus \mathtt{G}(\neg p))\right] \in \mathtt{LTL}\times\mathtt{K}.$$

We have $\mathfrak{L}, v_0, 0 \models \sigma$ if:

1. v_0 never satisfies p (neither today nor in the future);
2. at each timestamp i, $\sigma_{0,i}(v_0)$ has a neighbour w that does satisfy p at timestamp i, but never satisfies p in the future anymore.

The formula σ has the interesting property that for any pointed TG $(\mathfrak{G}, v_0, 0)$ such that $\mathfrak{G}, v_0, 0 \models \sigma$, we have $\mathrm{surf}(\mathfrak{G}) \geq \mathrm{len}(\mathfrak{G})$. In other words, any long TG model of σ must also be fat.

The reason is that at each new timestamp i, v_0 should have a neighbour that satisfies p, which must be different from its previous neighbours of timestamps $0, \ldots, i-1$ (since these previous neighbours cannot satisfy p anymore). To formally prove the above claim, one can show by induction on i that for all $i \in [\mathrm{len}(\mathfrak{G})]$,

$$|\{w \in V^{\mathfrak{G}} \mid \mathfrak{G}, w, i \models \mathtt{G}(\neg p)\}| \geq i + 1.$$

Taking $i = \mathrm{len}(\mathfrak{G}) - 1$ then yields the desired result immediately.

The property does not hold for QTGs; indeed, for all $n \in \mathbb{N}_1$, there exists a QTG model $\mathfrak{L}$ of σ such that $\mathrm{len}(\mathfrak{L}) = n$ and $\mathrm{surf}(\mathfrak{L}) = 2$. The idea of the construction is, at each timestamp i, to merge all the nodes that satisfy $\mathtt{G}(\neg p)$. The QTG $\mathfrak{L}$ can be defined as

- For all $i \in [n]$, the i-th graph $G_i^{\mathfrak{L}} = (V_i^{\mathfrak{L}}, E_i^{\mathfrak{L}}, L_i^{\mathfrak{L}})$ is defined by $V_i^{\mathfrak{L}} = \{v_0, w\}$, $E_i^{\mathfrak{L}} = \{(v_0, w)\}$, $L_i^{\mathfrak{L}}(v_0) = \varnothing$ and $L_i^{\mathfrak{L}}(w) = \{p\}$;
- For all $i \in [n-1]$, $\sigma_i(v_0) = \sigma_i(w) = v_0$ (this is where the merging occurs).

Let $\eta \in \mathtt{LTL}\times\mathtt{K}$. We define the *Fischer-Ladner closure* of η as the smallest set $\mathrm{cl}(\eta) \subset \mathtt{LTL}\times\mathtt{K}$ such that $\eta \in \mathrm{cl}(\eta)$ and that is closed under subformulae and unfoldings of the fixpoint operators $\mathtt{U}$ and $\mathtt{R}$; that is, for all $\varphi, \psi \in \mathtt{LTL}\times\mathtt{K}$,

- If $\varphi \bowtie \psi \in \mathrm{cl}(\eta)$ for $\bowtie \in \{\vee, \wedge\}$, then $\varphi, \psi \in \mathrm{cl}(\eta)$;
- If $\heartsuit\varphi \in \mathrm{cl}(\eta)$ for $\heartsuit \in \{\bigcirc, \ominus, \Diamond, \Box\}$, then $\varphi \in \mathrm{cl}(\eta)$;
- If $\varphi\,\mathtt{U}\,\psi \in \mathrm{cl}(\eta)$, then $\psi \vee (\varphi \wedge \bigcirc(\varphi\,\mathtt{U}\,\psi)) \in \mathrm{cl}(\eta)$;
- If $\varphi\,\mathtt{R}\,\psi \in \mathrm{cl}(\eta)$, then $\psi \wedge (\varphi \vee \ominus(\varphi\,\mathtt{R}\,\psi)) \in \mathrm{cl}(\eta)$.

The set $\mathrm{cl}(\eta)$ clearly contains all subformulae of η.

We define the size of an $\mathtt{LTL}\times\mathtt{K}$ formula as the size of its Fischer-Ladner closure: for $\eta \in \mathtt{LTL}\times\mathtt{K}$, $|\eta| = |\mathrm{cl}(\eta)|$. This definition might be surprising since usually in the literature of modal and temporal logics, the size of a formula is defined as the number of its subformulae. It is however not hard to show that $|\eta| = |\mathrm{cl}(\eta)|$ only exceeds the number of subformulae of η by a constant factor, so our definition is still equivalent to the conventions of the literature.

2.2. The temporarily-guarded μ-calculus $\mathtt{TG}\mu$

The *temporarily-guarded μ-calculus* $\mathtt{TG}\mu$ extends $\mathtt{LTL}\times\mathtt{K}$ by allowing to define formulae as solutions of explicit fixpoint equations. The combination of the restriction of temporal guardedness with the fact that time lines are finite (i.e. QTGs are *finite* sequences of graphs) ensures that fixpoints can only be unfolded a finite number of times (see Theorem 2.6). As a consequence, there is no need to distinguish between least and greatest fixpoints[6], so the syntax only contains one fixpoint binder operator $\mathtt{rec}$, instead of the usual μ and ν.

Definition 2.4 (Syntax of $\mathtt{TG}\mu$)

The syntax of the temporarily-guarded μ-calculus is inductively defined as follows:

$$\begin{aligned}\varphi ::= {} & \mid p \mid \neg p \mid X && (p \in \mathsf{AP},\ X \in \mathsf{Var}) \\ & \mid \varphi \vee \varphi \mid \varphi \wedge \varphi \\ & \mid \bigcirc\varphi \mid \ominus\varphi \\ & \mid \Diamond\varphi \mid \Box\varphi \\ & \mid \mathtt{rec}X.\varphi && (X \in \mathsf{Var})\end{aligned}$$

[6] This phenomenon (that least and greatest fixpoints coincide on guarded formulae interpreted on finite linear structures) is standard knowledge in the one-dimensional setting; see [Mat02] for a proof (that deals with the more general case of finite acyclic structures).

We write $\texttt{RTG}\mu$ for the set of formulae generated by the above grammar.

Note that the above set of formulae is not called $\texttt{TG}\mu$, but $\texttt{RTG}\mu$ (for *Raw* $\texttt{TG}\mu$). To define the actual syntax of $\texttt{TG}\mu$, we have to add some syntactical restrictions, which we present now.

Closeness. We can equip formulae of $\texttt{RTG}\mu$ with the standard notion of free variable (the operator $\texttt{rec}$ acts as a variable binder). We write $\mathrm{fv}(\varphi)$ for the set of free variables of a formula $\varphi \in \texttt{RTG}\mu$. For instance,

$$\mathrm{fv}(\texttt{rec}X.(Y \wedge \Diamond X)) = \{Y\} \quad \text{and} \quad \mathrm{fv}(X \vee \texttt{rec}X.\texttt{rec}Y.(X \vee Z)) = \{X, Z\}.$$

We say that a formula $\varphi \in \texttt{RTG}\mu$ is *closed* if $\mathrm{fv}(\varphi) = \varnothing$.

Temporal guardedness. We now present the syntactical restrition from which the logic takes its name. We say that a formula $\varphi \in \texttt{RTG}\mu$ is *temporarily-guarded* if every bound occurrence of a variable in φ is under the scope of a temporal modality ($\bigcirc$ or $\ominus$) within its binding fixpoint formula. For instance, of the four formulae

$$\varphi_1 = \texttt{rec}X.(p \vee \bigcirc X) \qquad \varphi_2 = \texttt{rec}X.\texttt{rec}Y.(Y \vee \bigcirc(X \wedge \Box p))$$
$$\varphi_3 = \texttt{rec}X.(p \vee \Diamond X) \qquad \varphi_4 = \texttt{rec}X.\bigcirc(\texttt{rec}Y.X \vee \ominus\Diamond Y),$$

only φ_1 and φ_4 are temporarily guarded: φ_2 is not because the only bound occurrence of Y is not under the scope of any modality, and φ_3 is not either because the only bound occurrence of X is not under the scope a temporal modality (although being under the scope of the spatial modality $\Diamond$).

The syntax of $\texttt{TG}\mu$ is then defined as the set of $\texttt{RTG}\mu$ formulae which are closed and temporarily-guarded:

$$\texttt{TG}\mu = \{\varphi \in \texttt{RTG}\mu \mid \varphi \text{ is closed and temporarily-guarded}\}.$$

Example 2.5
The formulae φ_1 and φ_4 defined above are closed, and therefore in $\texttt{TG}\mu$.

For the reader unfamiliar with the syntax of μ-calculi: a formula $\Phi = \texttt{rec}X.\varphi$ should be understood as the (unique) solution of the fixpoint equation $\Phi \equiv \varphi[\Phi/X]$. (In general, there exists multiple solutions, so we differentiate between the least solution $\Phi_{\text{least}} = \mu X.\varphi$ and the greatest solution $\Phi_{\text{greatest}} = \nu X.\varphi$; as explained in the introductory paragraph, the syntactical restrictions on $\texttt{TG}\mu$ ensure that solutions of these fixpoint equations are unique, so we only need a single fixpoint operator $\texttt{rec}$.)

As an example, the formula φ_1 is solution of the equation $\varphi_1 \equiv p \vee \bigcirc\varphi_1$. Its semantics is the same as the formula $\texttt{F}p \in \texttt{LTL}\times\texttt{K}$.

Given $(\texttt{rec}X.\varphi) \in \texttt{TG}\mu$, we define the *fixpoint unfolding formula* $\varphi[\texttt{rec}X.\varphi/X] \in \texttt{TG}\mu$ as the substitution of the variable X by the formula $\texttt{rec}X.\varphi$ in φ; it is obtained from φ by replacing every free[7] occurrence of X by $\texttt{rec}X.\varphi$. As an example, the fixpoint unfolding formula of $\varphi_1 \in \texttt{TG}\mu$ is

$$p \vee \bigcirc\varphi_1 = p \vee \bigcirc(\texttt{rec}X.(p \vee \bigcirc X)).$$

Note that because of temporal guardedness, all the (substituted) occurrences of $\texttt{rec}X.\varphi$ in $\varphi[\texttt{rec}X.\varphi/X]$ are under the scope of a temporal modality.

A very pleasant thing about the set $\texttt{TG}\mu$ is that it admits a natural well-founded induction principle, which is obtained by taking components subformulae of boolean and modal operators, and unfolding fixpoints. It essentially captures the fact as long as the temporal traces are finite (in the statement of Theorem 2.6, of length n), fixpoint formulae can only be unfolded a finite number of times.

Theorem 2.6 (Well-founded induction principle on $\texttt{TG}\mu$)

Let $n \in \mathbb{N}_1$ and $(\mathcal{H}(\varphi, i))_{(\varphi,i)\in\texttt{TG}\mu\times[n]}$ be a family of propositions. We suppose the following:

Base cases.

- For all $p \in \mathsf{AP}$ and $i \in [n]$, $\mathcal{H}(p, i)$ and $\mathcal{H}(\neg p, i)$ are true.
- For all $\varphi \in \texttt{TG}\mu$, $\mathcal{H}(\bigcirc\varphi, n-1)$ and $\mathcal{H}(\ominus\varphi, n-1)$ are true.

Inductive steps.

- For all $\varphi, \psi \in \texttt{TG}\mu$ and $i \in [n]$, if $\mathcal{H}(\varphi, i)$ and $\mathcal{H}(\psi, i)$ are true, then so are $\mathcal{H}(\varphi \vee \psi, i)$ and $\mathcal{H}(\varphi \wedge \psi, i)$.
- For all $\varphi \in \texttt{TG}\mu$ and $i \in [n-1]$, if $\mathcal{H}(\varphi, i+1)$ is true, then so are $\mathcal{H}(\bigcirc\varphi, i)$ and $\mathcal{H}(\ominus\varphi, i)$.
- For all $\varphi \in \texttt{TG}\mu$ and $i \in [n]$, if $\mathcal{H}(\varphi, i)$ is true, then so are $\mathcal{H}(\Diamond\varphi, i)$ and $\mathcal{H}(\Box\varphi, i)$.
- For all $(\texttt{rec}X.\varphi) \in \texttt{TG}\mu$ and $i \in [n]$, if $\mathcal{H}(\varphi[\texttt{rec}X.\varphi/X], i)$ is true, then so is $\mathcal{H}(\texttt{rec}X.\varphi, i)$.

Then $\mathcal{H}(\varphi, i)$ is true for all $(\varphi, i) \in \texttt{TG}\mu \times [n]$.

[7] Even though $(\texttt{rec}X.\varphi) \in \texttt{TG}\mu$, we only have $\varphi \in \texttt{RTG}\mu$, so free occurrences of X in φ can exist (see the example).

Proof. It suffices to prove that the binary relation $\to$ on $\mathtt{TG}\mu \times [n]$ defined by the exhaustive enumeration

$$\begin{array}{ll}
(\varphi \vee \psi, i) \to (\varphi, i) & (\varphi \wedge \psi, i) \to (\varphi, i) \\
(\varphi \vee \psi, i) \to (\psi, i) & (\varphi \wedge \psi, i) \to (\psi, i) \\
(\bigcirc \varphi, i) \to (\varphi, i+1) & (\ominus \varphi, i) \to (\varphi, i+1) \\
(\Diamond \varphi, i) \to (\varphi, i) & (\Box \varphi, i) \to (\varphi, i) \\
(\mathtt{rec}X.\varphi, i) \to (\varphi[\mathtt{rec}X.\varphi/X], i) &
\end{array}$$

terminates (Theorem 2.6 will then follow by the standard well-founded induction principle on terminating relations).

Let, for $\varphi \in \mathtt{TG}\mu$, $|\varphi|_t$ be the size of the syntax tree of φ, from which we removed all the subtrees rooted in a temporal modality (as an example, for the formulae $\varphi_1, \varphi_4 \in \mathtt{TG}\mu$ defined in the "Temporal guardedness" paragraph above, $|\varphi_1|_t = 3$ and $|\varphi_4|_t = 1$). We associate to the pair $(\varphi, i) \in \mathtt{TG}\mu \times [n]$ the "loop variant" $(n-1-i, |\varphi|_t) \in {\mathbb{N}_0}^2$ (where ${\mathbb{N}_0}^2$ is equipped with the lexicographic order). It is indeed a loop variant:

> **Claim 2.7**
> For $(\varphi, i), (\psi, j) \in \mathtt{TG}\mu \times [n]$, if $(\varphi, i) \to (\psi, j)$ then
>
> $$(n-1-i, |\varphi|_t) >_{\text{lex}} (n-1-j, |\psi|_t)$$
>
> (where $<_{\text{lex}}$ is the lexicographic order on ${\mathbb{N}_0}^2$).

Since $>_{\text{lex}}$ terminates (which is of course a rephrasing of the fact that the lexicographic order on ${\mathbb{N}_0}^2$ is well-founded), we must have that $\to$ terminates as well, which concludes the proof. ◀

This induction principle describes the "very essence" of temporal guardedness on finite sequences, which is that $\varphi[\mathtt{rec}X.\varphi/X]$ is actually a simpler formula than $\mathtt{rec}X.\varphi$ – just as, for instance, φ and ψ are simpler formulae than $\varphi \vee \psi$. The idea as that even though in general, $\varphi[\mathtt{rec}X.\varphi/X]$ is clearly a bigger formula than $\mathtt{rec}X.\varphi$, temporal guardedness ensure that the $\mathtt{rec}X.\varphi$ subformulae of the former are "pushed to the right" (in the sense that they shall be evaluated on a larger timestamp), until they reach the end of the sequence.

Semantics of $\mathtt{TG}\mu$. We are now ready to define the semantics of $\mathtt{TG}\mu$ formulae. Usually, the semantics of μ-calculi are defined as extensions $[\![\varphi]\!]$, making use of the Knaster-Tarski theorem [Kna28, Tar55] to define the semantics of the operators μ and ν as least and greatest fixpoints respectively [Koz83, DGL16]. The above induction principle enables us to give a much simpler definition of the semantics of $\mathtt{TG}\mu$ (all induction principles, in particular Theorem 2.6, come with the possibility to define objects by recursion on the set defined inductively, here $\mathtt{TG}\mu \times [n]$).

Definition 2.8 (Semantics of $\mathtt{TG}\mu$)

Let $\mathfrak{L}$ be a QTG of length n. We define the satisfaction relation $\models$ by well-founded induction on the pair $(\varphi, i) \in \mathtt{TG}\mu \times [n]$ (Theorem 2.6), as follows:

For all $(v, i) \in \mathrm{Pts}(\mathfrak{L})$,

$$
\begin{array}{lll}
\mathfrak{L}, v, i \models p & \text{iff} & p \in L_i^{\mathfrak{L}}(v) \\
\mathfrak{L}, v, i \models \neg p & \text{iff} & p \notin L_i^{\mathfrak{L}}(v) \\
\mathfrak{L}, v, i \models \varphi \vee \psi & \text{iff} & \mathfrak{L}, v, i \models \varphi \text{ or } \mathfrak{L}, v, i \models \psi \\
\mathfrak{L}, v, i \models \varphi \wedge \psi & \text{iff} & \mathfrak{L}, v, i \models \varphi \text{ and } \mathfrak{L}, v, i \models \psi \\
\mathfrak{L}, v, i \models \bigcirc \varphi & \text{iff} & i < n-1 \text{ and } \mathfrak{L}, v, i+1 \models \varphi \\
\mathfrak{L}, v, i \models \ominus \varphi & \text{iff} & i = n-1 \text{ or } (i < n-1 \text{ and } \mathfrak{L}, v, i+1 \models \varphi) \\
\mathfrak{L}, v, i \models \Diamond \varphi & \text{iff} & \exists w \in \mathrm{Nei}_i^{\mathfrak{L}}(v),\ \mathfrak{L}, w, i \models \varphi \\
\mathfrak{L}, v, i \models \Box \varphi & \text{iff} & \forall w \in \mathrm{Nei}_i^{\mathfrak{L}}(v),\ \mathfrak{L}, w, i \models \varphi \\
\mathfrak{L}, v, i \models \mathtt{rec}X.\varphi & \text{iff} & \mathfrak{L}, v, i \models \varphi[\mathtt{rec}X.\varphi/X].
\end{array}
$$

Example 2.9

1. As seen in Example 2.5, the formula $\mathtt{rec}X.(p \vee \bigcirc X) \in \mathtt{TG}\mu$ has the same semantics as $\mathtt{F}p$ (i.e. it says that in the future, the node on which we are will satisfy p).

2. The formula $\mathtt{rec}X.(p \vee \bigcirc\Diamond X) \in \mathtt{TG}\mu$ says that we can reach a node satisfying p by alternating temporal and spatial edges.

3. The formula $\mathtt{rec}X.\mathtt{rec}Y.((\Diamond p \wedge \bigcirc X) \vee \bigcirc Y) \in \mathtt{TG}\mu$ says that in the future, we should have a neighbour satisfying p infinitely often. Since by definition, every QTG has a finite length, this formula is not satisfiable.

As for $\mathtt{LTL}\times\mathtt{K}$, we define the Fischer-Ladner closure of a formula $\eta \in \mathtt{TG}\mu$ as the smallest set $\mathrm{cl}(\eta) \subset \mathtt{TG}\mu$ such that $\eta \in \mathrm{cl}(\eta)$ and for all formulae φ, ψ,

- If $\varphi \bowtie \psi \in \mathrm{cl}(\eta)$ for $\bowtie \in \{\vee, \wedge\}$, then $\varphi, \psi \in \mathrm{cl}(\eta)$;
- If $\heartsuit\varphi \in \mathrm{cl}(\eta)$ for $\heartsuit \in \{\bigcirc, \ominus, \Diamond, \Box\}$, then $\varphi \in \mathrm{cl}(\eta)$;
- If $\mathtt{rec}X.\varphi \in \mathrm{cl}(\eta)$, then $\varphi[\mathtt{rec}X.\varphi/X] \in \mathrm{cl}(\eta)$.

We also define the size of a $\mathtt{TG}\mu$ formula as the size of its Fischer-Ladner closure: for $\eta \in \mathtt{TG}\mu$, $|\eta| = |\mathrm{cl}(\eta)|$.

Remark 2.10 (Size measure of a μ-calculus formula)
Another natural definition of the size of a formula $\eta \in \mathtt{TG}\mu$ would be the number $|\mathrm{sub}(\eta)|$ of its subformulae. A surprising(?) fact about μ-calculi is that the size of the Fischer-Ladner closure of a formula is always *smaller* than the number of

its subformulae [DGL16]; so in our particular setting, we have that for all $\eta \in \mathtt{TG}\mu$, $|\mathrm{cl}(\eta)| \leq |\mathrm{sub}(\eta)|$. The reason is that the Fischer-Ladner closure $\mathrm{cl}(\eta)$ always identifies a fixpoint formula $\mathtt{rec}X.\varphi$ with the corresponding variable X, whereas they are counted separately as subformulae.

As an example (which shows that this inequality can be strict), we have

$$|\mathrm{cl}(\mathtt{rec}X.\bigcirc X)| = |\{\mathtt{rec}X.\bigcirc X, \bigcirc(\mathtt{rec}X.\bigcirc X)\}| = 2$$
$$|\mathrm{sub}(\mathtt{rec}X.\bigcirc X)| = |\{\mathtt{rec}X.\bigcirc X, \bigcirc X, X\}| = 3.$$

Actually, $|\mathrm{cl}(\eta)|$ can be *exponentially* smaller than $|\mathrm{sub}(\eta)|$: the paper [BFL15] defines a sequence of formulae $(\Phi_n)_{n\in\mathbb{N}_0}$ such that $|\mathrm{cl}(\Phi_n)|$ grows quadratically with n, while $|\mathrm{sub}(\Phi_n)|$ grows exponentially (see [DGL16]).

2.3. Expressiveness comparison

So far we have defined three logics interpreted on QTGs: $\mathtt{BTL}\times\mathtt{K}$ (which, as a reminder, is the fragment of $\mathtt{LTL}\times\mathtt{K}$ which does not contain the operators $\mathtt{U}$ and $\mathtt{R}$), $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$. In this section, we briefly discuss how these logics relate with each other in terms of expressiveness.

Given two logics $\mathcal{L}_1, \mathcal{L}_2 \in \{\mathtt{BTL}\times\mathtt{K}, \mathtt{LTL}\times\mathtt{K}, \mathtt{TG}\mu\}$, we write $\mathcal{L}_1 \sqsubseteq \mathcal{L}_2$ if $\mathcal{L}_2$ is *at least as expressive as* $\mathcal{L}_1$, that is,

$$\forall \varphi \in \mathcal{L}_1,\ \exists \psi \in \mathcal{L}_2,\ \varphi \equiv \psi$$

(where $\varphi \equiv \psi$ means that φ and ψ share the same truth values on all pointed QTGs). We write $\mathcal{L}_1 \sqsubset \mathcal{L}_2$ if $\mathcal{L}_2$ is *(strictly) more expressive than* $\mathcal{L}_1$, that is, $\mathcal{L}_1 \sqsubseteq \mathcal{L}_2$ and $\mathcal{L}_2 \not\sqsubseteq \mathcal{L}_1$.

First, since $\mathtt{BTL}\times\mathtt{K}$ is a fragment of $\mathtt{LTL}\times\mathtt{K}$, we clearly have $\mathtt{BTL}\times\mathtt{K} \sqsubseteq \mathtt{LTL}\times\mathtt{K}$. Another comparison which is less trivial but (standard and) still reasonably easy to obtain using the unfoldings of the operators $\mathtt{U}$ and $\mathtt{R}$, is that $\mathtt{LTL}\times\mathtt{K} \sqsubseteq \mathtt{TG}\mu$:

Lemma 2.11 (Embedding of $\mathtt{LTL}\times\mathtt{K}$ in $\mathtt{TG}\mu$)

For all $\varphi \in \mathtt{LTL}\times\mathtt{K}$, there exists $\psi \in \mathtt{TG}\mu$ such that $\varphi \equiv \psi$ and $|\varphi| = |\psi|$.

In particular, $\mathtt{LTL}\times\mathtt{K} \sqsubseteq \mathtt{TG}\mu$.

Proof. Let $X \in \mathsf{Var}$ be an arbitrary variable.

We define a function $\mathrm{Tr} : \mathtt{LTL}\times\mathtt{K} \to \mathtt{TG}\mu$ such that for all $\varphi \in \mathtt{LTL}\times\mathtt{K}$, $\varphi \equiv \mathrm{Tr}(\varphi)$

and $|\varphi| = |\mathrm{Tr}(\varphi)|$, by induction on φ, as follows.

$$\begin{array}{lll}
\mathrm{Tr}(q) & = q & \text{for } q \in \mathsf{AP} \cup \{\neg p,\ p \in \mathsf{AP}\} \\
\mathrm{Tr}(\varphi \bowtie \psi) & = \mathrm{Tr}(\varphi) \bowtie \mathrm{Tr}(\psi) & \text{for } \varphi, \psi \in \mathtt{LTL}{\times}\mathtt{K} \text{ and } \bowtie \in \{\vee, \wedge\} \\
\mathrm{Tr}(\heartsuit \varphi) & = \heartsuit \mathrm{Tr}(\varphi) & \text{for } \varphi \in \mathtt{LTL}{\times}\mathtt{K} \text{ and } \heartsuit \in \{\bigcirc, \ominus, \Diamond, \Box\} \\
\mathrm{Tr}(\varphi \,\mathtt{U}\, \psi) & = \mathtt{rec}X.(\mathrm{Tr}(\psi) \vee (\mathrm{Tr}(\varphi) \wedge \bigcirc X)) & \text{for } \varphi, \psi \in \mathtt{LTL}{\times}\mathtt{K} \\
\mathrm{Tr}(\varphi \,\mathtt{R}\, \psi) & = \mathtt{rec}X.(\mathrm{Tr}(\psi) \wedge (\mathrm{Tr}(\varphi) \vee \ominus X)) & \text{for } \varphi, \psi \in \mathtt{LTL}{\times}\mathtt{K}.
\end{array}$$

Taking $\psi = \mathrm{Tr}(\varphi)$ gives the announced result. ◀

Gathering the above results, we obtain the following hierarchy: $\mathtt{BTL}{\times}\mathtt{K} \sqsubseteq \mathtt{LTL}{\times}\mathtt{K} \sqsubseteq \mathtt{TG}\mu$.

Separation results. The natural question is then whether this hierarchy collapses or is strict. It is standard knowledge (see e.g. [DGL16]) that $\mathtt{BTL} \sqsubset \mathtt{LTL} \sqsubset \mathtt{LT}\mu$ (where $\mathtt{LT}\mu$ is the linear-time μ-calculus); specifically, there is no BTL formula equivalent (on words) to $\mathtt{F}p \in \mathtt{LTL}$, and no LTL formula equivalent (on words) to $\mu X.(p \vee \bigcirc \bigcirc X) \in \mathtt{LT}\mu$ (this formula says that p should hold at an even timestamp).

With a small trick, we can deduce from these results that $\mathtt{BTL}{\times}\mathtt{K} \sqsubset \mathtt{LTL}{\times}\mathtt{K} \sqsubset \mathtt{TG}\mu$. Let us prove that $\mathtt{BTL}{\times}\mathtt{K} \sqsubset \mathtt{LTL}{\times}\mathtt{K}$: suppose, by contradiction, that there exists $\eta \in \mathtt{BTL}{\times}\mathtt{K}$ such that $\eta \equiv_{\mathrm{QTGs}} \mathtt{F}p$ (here, $\mathtt{F}p$ is seen as a formula of $\mathtt{LTL}{\times}\mathtt{K}$). In particular, $\eta \equiv_{\mathrm{Words}} \mathtt{F}p$ (a word is seen as a special case of QTG, where the graph at each timestamp contains a single node). Let $\vartheta \in \mathtt{BTL}$ be obtained from η by replacing each subformula of the form $\Diamond\varphi$ by $\mathtt{ff}$, and each subformula of the form $\Box\varphi$ by $\mathtt{tt}$. Then $\eta \equiv_{\mathrm{Words}} \vartheta$, so by transitivity of $\equiv_{\mathrm{Words}}$, we obtain $\vartheta \equiv_{\mathrm{Words}} \mathtt{F}p$, hence the contradiction (no BTL formula is equivalent on words to $\mathtt{F}p$). The same trick proves that no $\mathtt{LTL}{\times}\mathtt{K}$ formula is equivalent (on QTGs) to $\mathtt{rec}X.(p \vee \bigcirc \bigcirc X) \in \mathtt{TG}\mu$, so that $\mathtt{LTL}{\times}\mathtt{K} \sqsubset \mathtt{TG}\mu$. We can therefore conclude:

Proposition 2.12 (The hierarchy is strict)

We have the following expressiveness comparisons: $\mathtt{BTL}{\times}\mathtt{K} \sqsubset \mathtt{LTL}{\times}\mathtt{K} \sqsubset \mathtt{TG}\mu$.

2.4. Bisimilarity and elementary equivalence

Let $\mathcal{L} \in \{\mathtt{BTL}{\times}\mathtt{K}, \mathtt{LTL}{\times}\mathtt{K}, \mathtt{TG}\mu\}$ be a logical language. Given two pointed QTGs $(\mathfrak{A}, v, i)$ and $(\mathfrak{B}, w, j)$, we write $(\mathfrak{A}, v, i) \equiv_{\mathcal{L}} (\mathfrak{B}, w, j)$ if $(\mathfrak{A}, v, i)$ and $(\mathfrak{B}, w, j)$ are elementary equivalent with respect to $\mathcal{L}$, that is,

$$\forall \varphi \in \mathcal{L},\ (\mathfrak{A}, v, i \models \varphi \quad \text{iff} \quad \mathfrak{B}, w, j \models \varphi).$$

On (static) graphs, it is standard knowledge that bisimilarity characterises elementary equivalence[8] for any reasonable (one-dimensonal) modal logic (notably the basic modal

[8] As long as the involved graphs are finitely branching. In this work, we only consider finite structures, so finite branching is always automatically verified.

logic K and the modal μ-calculus $\mathcal{L}_\mu$, see e.g. [DGL16] for proofs). These results are often referred to as *Hennessy-Milner properties*. The following proposition states that all the logics considered in this document have the Hennessy-Milner property w.r.t. spatio-temporal bisimulation.

Proposition 2.13 (Hennessy-Milner property)

Let $(\mathfrak{A}, v, i)$ and $(\mathfrak{B}, w, j)$ be two pointed QTGs. The following are equivalent:

i) $(\mathfrak{A}, v, i) \rightleftarrows (\mathfrak{B}, w, j)$;

ii) $(\mathfrak{A}, v, i) \equiv_{\mathtt{BTL}\times\mathtt{K}} (\mathfrak{B}, w, j)$;

iii) $(\mathfrak{A}, v, i) \equiv_{\mathtt{LTL}\times\mathtt{K}} (\mathfrak{B}, w, j)$;

iv) $(\mathfrak{A}, v, i) \equiv_{\mathtt{TG}\mu} (\mathfrak{B}, w, j)$;

We do not write the proof here, since it is exactly the same as for the standard bisimulation on graphs. The interested reader can find a detailed proof in the TIME paper (see the Introduction section).

Let η be a formula in any of the three logics $\mathtt{BTL}\times\mathtt{K}$, $\mathtt{LTL}\times\mathtt{K}$ or $\mathtt{TG}\mu$. We say that η is *satisfiable in a QTG* if there exists a pointed QTG $(\mathfrak{L}, v, i)$ such that $\mathfrak{L}, v, i \models \eta$. Similarly, we say that η is *satisfiable in a TG* if there exists a pointed TG $(\mathfrak{G}, v, i)$ such that $\mathfrak{G}, v, i \models \eta$.

Clearly, if η is satisfiable in a TG, then it is also in a QTG (since any TG is, in particular, a QTG). The converse is also true, by Theorem 1.7 and Proposition 2.13. Therefore, we will simply say that η is *satisfiable* if it is, equivalently, satisfiable in a QTG or a TG.

Remark 2.14 (Consequences on proof theory)

Let $\mathcal{L} \in \{\mathtt{BTL}\times\mathtt{K}, \mathtt{LTL}\times\mathtt{K}, \mathtt{TG}\mu\}$ be a logic. This document studies two classes of models for the formulae in $\mathcal{L}$, QTGs and TGs. What we saw above is that a formula $\eta \in \mathcal{L}$ is satisfiable in the class of QTGs iff it is in the class of TGs. In other words, moving from the class of TGs to the (larger) class of QTGs to interpret $\mathcal{L}$ does not enable us to satisfy more formulae.

Dually, a formula $\eta \in \mathcal{L}$ is valid in the class of QTGs iff it is in the class of TGs. In other words, moving from the class of QTGs to the (smaller) class of TGs to interpret $\mathcal{L}$ does not enable us to prove more things (i.e. to make more formulae valid). The proof theory of $\mathcal{L}$ is therefore the same, whether we interpret it on QTGs or TGs (as a consequence, any proof system that characterises validity in one class will also characterise validity for the other).

However, since the translation from QTGs to TGs (Theorem 1.7) increases the surface, QTGs can be seen as more *concise* than TGs, in the sense that if a formula $\eta \in \mathcal{L}$ is satisfiable, then it might have QTG models that are smaller (in the spatial

dimension) than any TG model – or dually, if a formula $\eta \in \mathcal{L}$ is not valid, then it might have QTG counterexamples that are smaller (in the spatial dimension) than any TG counterexample. We have already seen this phenomenon in Example 2.3. It will appear more clearly in Corollary 4.5, and especially in Theorem 5.2, in which we define $\mathtt{LTL}\times\mathtt{K}$ formulae that admit QTG models whose surface is exponentially smaller than the surface of any TG model.

2.5. The philosophy of QTGs

We have now defined all the objects that we seek to understand; namely, (Q)TGs and the logics $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$. Before heading to the main technical developments and results of the document, let us do a little break and discuss the "status" that our newly introduced structure of QTG has in the theory.

First, what we have already seen (notably in Theorem 1.7, Example 2.3 and Remark 2.14) is that QTGs can be used as a compact way to represent TGs. Said differently, a QTG can be seen as a compressed TG, where the compression consists in merging nodes at a same timestamp that we consider to be equivalent[9], and deleting the past of certain nodes when we consider it to be irrelevant[10]. Theorem 1.7 is then nothing but the corresponding decompression procedure.

Besides, as we shall see in the next sections, QTGs have an importance at the technical level. What we mean by that is that even if we decide to ignore them and reason only on TGs for the small model property and the satisfiability algorithm, they will still appear naturally at some point in the proof (this specific point is Theorem 3.5, see the discussion that follows it).

We however believe that QTGs are not only concise representations of TGs, and not a mere technical artefact in proofs either. On the contrary, we think that QTGs have a conceptual relevance in the theory, and even that they form a more natural class of models for our spatio-temporal modal logics than TGs.

To explain why, we shall first observe a common property of modal (and temporal) logics: *in general, modal (and temporal) logics are not graded*, i.e. a formula of a modal logic can never enforce to create two different nodes that have exactly the same role[11]. Results that embody the nongradedness of modal logics often take this form: "If we have a formula φ, a model $\mathfrak{M} \models \varphi$ and two nodes $v, v' \in \mathfrak{M}$ that have exactly the same role w.r.t. φ,[12] then the structure $\mathfrak{M}'$ obtained by merging v and v' is also a model of φ". In $\mathtt{K}$, the correctness of filtration [DGL16] is such a result. In the linear-time setting, we do not merge v and v', but instead cut everything between them (including one of them):

[9] This corresponds to the possible non-injectivity of the temporal successor functions.

[10] This corresponds to the possible non-surjectivity of the temporal successor functions.

[11] On the contrary, first-order logic *is* graded: as an example, the formula $\exists \mathtt{x}.\exists \mathtt{y}.(\mathtt{x} \neq \mathtt{y} \wedge \varphi(\mathtt{x}) \wedge \varphi(\mathtt{y}))$ forces to have two different nodes that both satisfy a formula φ.

[12] This property is usually formalised by the fact that v and v' are elementary equivalent w.r.t. the set of subformulae, or the Fischer-Ladner closure, of φ.

as an example, LTL has the property that "If we have a formula $\varphi \in \mathtt{LTL}$, a linear-time structure w such that $w, 0 \models \varphi$ and two timestamps i, j that have exactly the same role w.r.t. φ, then we also have $w_{[0,\dots,i-1,j,\dots]}, 0 \models \varphi$" (see Lemma 6.3.2 in [DGL16]).[13]

Our spatio-temporal logics, when interpreted on TGs, do have a form of gradedness. As an example, the formula

$$p \wedge \bigcirc\varphi \wedge \Diamond(\neg p \wedge \bigcirc\varphi)$$

forces to create two different nodes that both satisfy φ. They cannot be merged, since it would mean also merging them at the previous timestamp (and at the previous timestamp, one satisfies p and the other satisfies $\neg p$, so the merging will not work). This supports the idea that TGs are not "good" models to interpret our spatio-temporal logics. On the contrary, if we allow QTGs, the gradedness phenomenon disappears. Notably, the above formula has a QTG model that only has one node satisfying φ (which is given by merging the two nodes that satisfy φ in the TG described above – this is authorized by the possible non-injectivity of the temporal successor functions).

3. Spatio-temporal automata

Spatio-temporal automata, which we shall introduce in this section, are automata that take pointed QTGs as inputs. They therefore have the typing "pointed QTG $\to$ Bool", which is the same as the semantics of formulae in our spatio-temporal logics.

In Section 3.1, we define automata and their semantics. Section 3.2 studies the problem, given a spatio-temporal automaton $\mathcal{A}$, of deciding whether $\mathrm{L}(\mathcal{A}) \neq \varnothing$ (this is of course the analogue of the satisfiability problem of a formula in a spatio-temporal logic). Finally, in Section 3.3, we discuss more compact ways to represent an automaton in a computer or an algorithm.

3.1. Definition

In all this section, we note $\Sigma = \mathcal{P}(\mathsf{AP})$.

Definition 3.1 (Spatio-temporal automaton)

A *spatio-temporal automaton* is a tuple $\mathcal{A} = (Q, \delta_s, \delta_t, \lambda, I, F)$, where:

- Q is a finite set of *states*;
- $\delta_s \subset Q \times \mathcal{P}(Q)$ is the *spatial transition relation*;
- $\delta_t \subset Q \times Q$ is the *temporal transition relation*;
- $\lambda : Q \to \mathcal{P}(\Sigma)$ specifies the possible *labelings* of each state;

[13] A careful reader will notice that the ideas of 1. merging the nodes in a graph that are equivalent, and 2. cutting everything inbetween timestamps that are equivalent, are the main ingredients of the proof of Theorem 3.5, which is itself at the core of the proof of the small model property.

- $I, F \subset Q$ are the set of *initial* and *final* states, respectively.

A spatio-temporal automaton contains two transition functions: a spatial one δ_s, that behaves similarly to the one of a tree automaton ($(q, N) \in \delta_s$ means that the neighbours of a node processed by the state q may be processed by the set of states N), and a temporal one, that behaves similarly to a standard (nondeterministic) word automaton.

The following definition explains how a QTG is processed by a spatio-temporal automaton.

Definition 3.2 (Language of a spatio-temporal automaton)

Let $\mathcal{A} = (Q, \delta_s, \delta_t, \lambda, I, F)$ be an automaton and $(\mathfrak{L}, v_0, 0)$ be a pointed QTG of length n. An *accepting run* of $(\mathfrak{L}, v_0, 0)$ in $\mathcal{A}$ is a tuple $\rho = (\rho_i)_{i\in[n]} \in \prod_{i=0}^{n-1} Q^{V_i^{\mathfrak{L}}}$ such that:

- $\forall i \in [n],\ \forall v \in V_i^{\mathfrak{L}},\ (\rho_i(v), \rho_i(\mathrm{Nei}_i^{\mathfrak{L}}(v))) \in \delta_s$;
- $\forall i \in [n-1],\ \forall v \in V_i^{\mathfrak{L}},\ (\rho_i(v), \rho_{i+1}(\sigma_i^{\mathfrak{L}}(v))) \in \delta_t$;
- $\forall i \in [n],\ \forall v \in V_i^{\mathfrak{L}},\ L_i^{\mathfrak{L}}(v) \in \lambda(\rho_i(v))$;
- $\rho_0(v_0) \in I$;
- $\rho_{n-1}(V_{n-1}^{\mathfrak{L}}) \subset F$.

We say that $(\mathfrak{L}, v_0, 0)$ is *accepted* by $\mathcal{A}$ if there exists an accepting run of $(\mathfrak{L}, v_0, 0)$ in $\mathcal{A}$. We call *language of* $\mathcal{A}$ the class

$$\mathrm{L}(\mathcal{A}) = \{(\mathfrak{L}, v_0, 0) \text{ pointed QTG} \mid (\mathfrak{L}, v_0, 0) \text{ is accepted by } \mathcal{A}\}.$$

In the rest of the document, we will not deal with other types of automata, so we will simply say "automaton" for a spatio-temporal automaton.

Example 3.3
The automaton $\mathcal{A}_{\text{even}} = (\{0,1\}, \delta_s, \delta_t, \lambda, \{0\}, \{1\})$, where

$$\begin{aligned}\delta_s &= \{(0, \varnothing), (0, \{0\}), (1, \varnothing), (1, \{1\})\}\\ \delta_t &= \{(0,1), (1,0)\}\\ \lambda(0) &= \lambda(1) = \Sigma\end{aligned}$$

recognises the class of pointed QTGs with even length. It is therefore equivalent to the formula $\mathtt{rec}X.(\mathtt{tt} \vee \bigcirc\bigcirc X) \in \mathtt{TG}\mu$, in the sense that for all pointed QTGs $(\mathfrak{L}, v_0, 0)$, we have

$$(\mathfrak{L}, v_0, 0) \in \mathrm{L}(\mathcal{A}_{\text{even}}) \quad \text{iff} \quad \mathfrak{L}, v_0, 0 \models \mathtt{rec}X.(\mathtt{tt} \vee \bigcirc\bigcirc X).$$

We will see later on (Theorem 4.3) that for any formula $\eta \in \mathtt{TG}\mu$, we can build an automaton $\mathcal{A}^\eta$ that is equivalent to η.

Example 3.4
Let $\mathsf{AP} = \{p\}$. The automaton $(Q, \delta_s, \delta_t, \lambda, I, F)$, where

$$
\begin{aligned}
Q &= \{\mathtt{has}p, \mathtt{any}\} \\
\delta_s &= \{(\mathtt{has}p, \varnothing), (\mathtt{has}p, \{\mathtt{has}p\})\} \cup \{(\mathtt{any}, N),\ N \subset Q\} \\
\delta_t &= \{(\mathtt{has}p, \mathtt{any}), (\mathtt{any}, \mathtt{any})\} \\
\lambda(\mathtt{has}p) &= \{\{p\}\} \\
\lambda(\mathtt{any}) &= \{\varnothing, \{p\}\} = \Sigma \\
I &= \{\mathtt{has}p\} \\
F &= \{\mathtt{has}p, \mathtt{any}\} = Q
\end{aligned}
$$

recognises the class of pointed QTGs $(\mathfrak{L}, v_0, 0)$ such that in the first graph $G_0^{\mathfrak{L}}$, any node reachable from v_0 satisfies p.

Example 3.4 shows that, on the contrary to $\mathtt{TG}\mu$ formulae, automata *can* explore a single graph to an arbitrary depth, at least to satisfy a satefy condition. We will however see later on (Proposition 4.6) that automata cannot express spatial reachability conditions.

3.2. The nonemptiness test

In this section, we are interested, given a spatio-temporal automaton $\mathcal{A}$, in determining if $\mathrm{L}(\mathcal{A}) \neq \varnothing$. This problem is of course the analogue of the satisfiability problem of a formula interpreted on pointed QTGs. The following theorem gives two characterisations of the nonemptiness of $\mathrm{L}(\mathcal{A})$. The first one (Point ii)) says that any automaton whose language is nonempty accepts a QTG whose size can be controlled; hence, it is a kind of small model property for automata[14]. The second one (Point iii)) is a characterisation that only depends on the inner structure of $\mathcal{A}$; we will use it to design an algorithm that decides the nonemptiness problem.

Theorem 3.5 (Nonemptiness test)

Let $\mathcal{A} = (Q, \delta_s, \delta_t, \lambda, I, F)$ be an automaton. The following are equivalent:

i) $\mathrm{L}(\mathcal{A}) \neq \varnothing$;

ii) There exists $(\mathfrak{L}, v_0, 0) \in \mathrm{L}(\mathcal{A})$ such that $\mathrm{len}(\mathfrak{L}) \leq 2^{|Q|}$ and $\mathrm{surf}(\mathfrak{L}) \leq |Q|$;

iii) There exists $n \in [\![1, 2^{|Q|}]\!]$ and a sequence $(Q_i)_{i \in [n]} \in \mathcal{P}(Q)^n$ such that:
- $\forall i \in [n],\ \forall q \in Q_i,\ \exists N \subset Q_i,\ (q, N) \in \delta_s$;
- $\forall i \in [n-1],\ \forall q \in Q_i,\ \exists r \in Q_{i+1},\ (q, r) \in \delta_t$;

[14] It will in fact be used in Section 4 to show the small model property of $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$.

- $\forall i \in [n],\ \forall q \in Q_i,\ \lambda(q) \neq \varnothing$;
- $Q_0 \cap I \neq \varnothing$;
- $Q_{n-1} \subset F$.

Proof. Let iii') be as iii), except that we only demand $n \in \mathbb{N}_1$ (instead of $n \in [\![1, 2^{|Q|}]\!]$). We show the circular implications i) $\Rightarrow$ iii$'$) $\Rightarrow$ iii) $\Rightarrow$ ii) $\Rightarrow$ i).

i) $\Rightarrow$ iii$'$) We suppose i): let $(\mathfrak{L}, v_0, 0) \in \mathrm{L}(\mathcal{A})$ and ρ be an accepting run of $(\mathfrak{L}, v_0, 0)$ in $\mathcal{A}$. Let $n = \mathrm{len}(\mathfrak{L})$ and for all $i \in [n]$, $Q_i = \rho_i(V_i^{\mathfrak{L}})$. It is easily verified that the sequence $(Q_i)_{i\in[n]}$ satisfies the requirements of iii').

iii$'$) $\Rightarrow$ iii) To show that we can ensure that $n \leq 2^{|Q|}$ in iii'), we eliminate the cycles in the path $Q_0 \rightsquigarrow \cdots \rightsquigarrow Q_{n-1}$.

More precisely, suppose iii') and let n be minimal such that there exists a sequence $(Q_0, \ldots, Q_{n-1})$ that satisfies the requirements of iii'). We suppose, by contradiction, that $n > 2^{|Q|}$. By the pigeonhole principle, there exists $0 \leq i < j \leq n-1$ such that $Q_i = Q_j$. Then the sequence $(Q_0, \ldots, Q_{i-1}, Q_j, \ldots, Q_{n-1})$ also satisfies the requirements of iii'), and has length $n-(j-i) < n$, which contradicts the minimality of n.

Therefore, we must have $n \leq 2^{|Q|}$, which gives iii).

iii) $\Rightarrow$ ii) We suppose iii): let $n \in [\![1, 2^{|Q|}]\!]$, and $(Q_i)_{i\in[n]}$ be as in iii). We use these sets to bring to life a pointed QTG accepted by $\mathcal{A}$.

Let us fix, for all $i \in [n]$ and $q \in Q_i$, some set $N_i(q) \subset Q_i$ such that $(q, N_i(q)) \in \delta_s$. Similarly, we fix some $L_i(q) \in \lambda(q) (\neq \varnothing)$, and if $i < n-1$, some $\sigma_i(q) \in Q_{i+1}$ such that $(q, \sigma_i(q)) \in \delta_t$.

We then define the QTG

$$\mathfrak{Q} = (n, (G_i = (Q_i, E_i, L_i))_{i\in[n]}, (\sigma_i)_{i\in[n]}),$$

where for all $i \in [n]$, $E_i = \{(q, r) \in Q_i^2 \mid r \in N_i(q)\}$.

Let $q_0 \in Q_0 \cap I$. It is easy verification that $(\mathrm{id}_{Q_i})_{i\in[n]}$ is an accepting run of $(\mathfrak{Q}, q_0, 0)$ in $\mathcal{A}$; hence, $(\mathfrak{Q}, q_0, 0) \in \mathrm{L}(\mathcal{A})$. Moreover, $\mathrm{len}(\mathfrak{Q}) = n \leq 2^{|Q|}$ and $\mathrm{surf}(\mathfrak{Q}) = \max_{i\in[n]} |Q_i| \leq |Q|$.

ii) $\Rightarrow$ i) is clear. ◀

The proof of Theorem 3.5 can be turned into a procedure to "compress" a given element of $\mathrm{L}(\mathcal{A})$, that we may describe in two steps. First, for each timestamp i, whenever several nodes of the i-th graph $G_i^{\mathfrak{L}}$ of $\mathfrak{L}$ are mapped to the same state q, we choose an arbitrary one v, delete all the others, and redirect the incoming spatial and temporal edges of the deleted nodes to v. Second, whenever two graphs $G_i^{\mathfrak{L}}$, $G_j^{\mathfrak{L}}$ (for $i < j$) are mapped to the same states, we cut everything in between, and are able to reconnect them. The output of this compression procedure is a QTG in $\mathrm{L}(\mathcal{A})$ that has length $\leq 2^{|Q|}$ and surface

$\leq |Q|$.

Note that even if the input of the procedure is a TG, the output may only be a QTG (this typically happens if there are two nodes v, w such that $\rho_i(v) \neq \rho_i(w)$ and $\rho_{i+1}(v) = \rho_{i+1}(w)$). It is actually this fact that mathematically motivates the introduction of QTGs: in this specific step, it is easy to build a QTG, but not directly a TG.

Deciding the nonemptiness problem. Using Theorem 3.5 Point iii), we can build an algorithm to decide, given an automaton $\mathcal{A}$, if $\mathrm{L}(\mathcal{A}) \neq \varnothing$. This algorithm is nondeterministic and runs with space $\mathcal{O}(|Q|)$ (where Q is the set of states of $\mathcal{A}$).

We first (nondeterministically) guess $1 \leq n \leq 2^{|Q|}$. We then guess the sets Q_i for an increasing i, and check that they satisfy the conditions of Theorem 3.5 Point iii). At step i of the guessing, we only need to store Q_i and Q_{i+1} ($Q_0, \ldots, Q_{i-1}$ can be thrown away), that we only need a space $\mathcal{O}(|Q|)$ to represent.

Remark 3.6 (A deterministic algorithm)
We can actually design a *deterministic* algorithm that runs with space $\mathcal{O}(|Q|)$, to decide the nonemptiness problem. (This algorithm will not be necessary for the rest of the document, so we only sketch the construction here.)

Let $\mathcal{A}$ be an automaton; we seek to determine if $\mathrm{L}(\mathcal{A}) \neq \varnothing$. Let, for $n \in \mathbb{N}_1$, $C_n \subset Q$ be the set of states q such that $\mathrm{L}(\mathcal{A}[I \leftarrow q])$ contains some pointed QTG of length n (where $\mathcal{A}[I \leftarrow q] = (Q, \delta_s, \delta_t, \lambda, \{q\}, F)$ is as $\mathcal{A}$, except that the only initial state is q). We clearly have

$$\mathrm{L}(\mathcal{A}) \neq \varnothing \quad \text{iff} \quad \exists n \in \mathbb{N}_1,\ C_n \cap I \neq \varnothing.$$

Hence, to decide the nonemptiness problem for $\mathcal{A}$, it suffices to compute the sets C_n.

The sets C_n can actually be computed by induction on n, using the following coinductive characterisations:

- $C_0 \subset Q$ is the biggest set such that $C_0 \subset F$ and for all $q \in C_0$, we have 1. $\lambda(q) \neq \varnothing$ and 2. there exists $N \subset C_0$ such that $(q, N) \in \delta_s$.
- For $n \in \mathbb{N}_1$, $C_{n+1} \subset Q$ is the biggest set such that for all $q \in C_{n+1}$, we have 1. $\lambda(q) \neq \varnothing$, 2. there exists $N \subset C_{n+1}$ such that $(q, N) \in \delta_s$ and 3. there exists $r \in C_n$ such that $(q, r) \in \delta_t$.

The procedure is then the following. We successively build the sets C_n for an increasing $n \in [\![1, 2^{|Q|}]\!]$, using the above characterisations. If at some iteration n of the loop, we find out that $C_n \cap I \neq \varnothing$, then we return True; else, we return False. At iteration n of the loop, we only need to store C_n and C_{n-1} ($C_0, \ldots, C_{n-2}$ can be thrown away), that we only need a space $\mathcal{O}(|Q|)$ to represent.

Remark 3.7 (The nonemptiness problem is in PSpace)
Since $|Q| \leq |\mathcal{A}|$, we have in particular that the nonemptiness problem for spatio-temporal automata is in PSPACE (either directly with the algorithm of Remark 3.6, or with the simple nondeterministic algorithm and Savitch's theorem).

However, in general, $|\mathcal{A}|$ is not polynomial in $|Q|$ (because δ_s might need a space exponential in $|Q|$ to be stored – this issue is specifically addressed in Section 3.3), so our claim that the nonemptiness problem can be solved with space $\mathcal{O}(|Q|)$ is actually stronger than saying that it is in PSPACE.

3.3. Representing a spatio-temporal automaton in a computer

Given Definition 3.1, the most natural way to represent an automaton $\mathcal{A}$ is to store δ_s and δ_t as lists of pairs $(q, N) \in Q \times \mathcal{P}(Q)$ and $(q, r) \in Q \times Q$ respectively, and λ as a table of length $|Q|$ with entries in $\mathcal{P}(\Sigma)$. This representation can be very unefficient: notably, in the worst case, δ_s needs a space $|Q|2^{|Q|}$ to be stored, and λ needs a space $|Q||\Sigma| = |Q|2^{|\mathsf{AP}|}$. In the end, $|\mathcal{A}|$ might be exponential in both $|Q|$ and $|\mathsf{AP}|$.

This can lead to troublesome effects on complexity: any algorithm that requires in a subroutine to build a spatio-temporal automaton might see its spatial complexity blow up just because of an unefficient representation of δ_s and λ. Such a situation will actually happen in Section 4.3, where we describe an algorithm to decide satisfiability in $\mathtt{TG}\mu$: given an input formula $\eta \in \mathtt{TG}\mu$, the procedure requires to build a spatio-temporal automaton $\mathcal{A}^\eta$, whose number of states is exponential in $|\eta|$ but whose size, if $\mathcal{A}^\eta$ is represented as above, becomes doubly-exponential in $|\eta|$. We describe below alternative ways of representing δ_s and λ, that we will use in Section 4.3 to store $\mathcal{A}^\eta$ with a space only (singly-)exponential in $|\eta|$ (leading to an algorithm in EXPSPACE to decide satisfiability in $\mathtt{TG}\mu$).

Representing δ_s. Let $\mathcal{B}(Q)$ be the set of formulae of propositional logic built over Q; that is, $\mathcal{B}(Q)$ is defined by the grammar

$$\varphi ::= q \mid \neg q \mid \varphi \vee \varphi \mid \varphi \wedge \varphi \quad (q \in Q).$$

Formulae in $\mathcal{B}(Q)$ are interpreted on subsets of Q: for $N \subset Q$, we define $N \models q$ iff $q \in N$, and $N \models \neg q$ iff $q \notin N$ (and the semantics of the boolean operators are defined the natural way). As an example, $N \models (q_1 \wedge q_2) \vee (q_3 \wedge \neg q_4)$ iff either $\{q_1, q_2\} \subset N$, or $q_3 \in N$ and $q_4 \notin N$.

The temporal transition relation can then be represented by a function $\varepsilon_s : Q \to \mathcal{B}(Q)$: for $q \in Q$, $\varepsilon_s(q)$ specifies all the sets $N \subset Q$ such that (q, N) is in the temporal transition relation. Formally, such a function ε_s induces the temporal transition relation

$$\delta_s = \{(q, N) \in Q \times \mathcal{P}(Q) \mid N \models \varepsilon_s(q)\}.$$

Representing λ. To represent λ, we proceed the same way, but we replace $\mathcal{P}(Q)$ by $\mathcal{P}(\mathsf{AP})$.[15] Let $\mathcal{B}(\mathsf{AP})$ be the set of formulae of propositional logic built over AP; that is, $\mathcal{B}(\mathsf{AP})$ is defined by the grammar

$$\varphi ::= p \mid \neg p \mid \varphi \vee \varphi \mid \varphi \wedge \varphi \quad (p \in \mathsf{AP}).$$

Formulae in $\mathcal{B}(\mathsf{AP})$ are interpreted on labelings $L \in \Sigma (= \mathcal{P}(\mathsf{AP}))$: for $L \in \Sigma$, we define $L \models p$ iff $p \in L$, and $L \models \neg p$ iff $p \notin L$.

The possible labelings of the states can then be represented by a function $\mu : Q \to \mathcal{B}(\mathsf{AP})$. Specifically, such a function μ induces the labling of the states $\lambda : Q \to \mathcal{P}(\Sigma)$ defined by

$$\forall q \in Q,\ \lambda(q) = \{L \in \Sigma \mid L \models \mu(q)\}.$$

These representations do not enable to store an automaton more efficiently in the general case. However, for the specific case of Section 4.3, it will indeed enable to store $\mathcal{A}^\eta$ with a space only singly-exponential in $|\eta|$.

4. Small model property

Now that the stage is set, we can tackle the main results of our work: the small model property and the algorithm to decide satisfiability in $\mathtt{LTL}{\times}\mathtt{K}$ and $\mathtt{TG}\mu$.

The strategy consists, given a formula $\eta \in \mathtt{TG}\mu$, in building an equivalent spatio-temporal automaton $\mathcal{A}^\eta$ (Theorem 4.3). We can then transfer what we proved in Section 3.2 to formulae in $\mathtt{TG}\mu$, resulting in the desired results.

4.1. From $\mathtt{TG}\mu$ to automata

This section aims at building the automaton $\mathcal{A}^\eta$. The construction is quite similar to e.g. the standard translation from $\mathtt{LTL}$ to Büchi automata [VW86, DGL16] (our case is actually even simpler, because the finiteness of the traces enables an easy final condition, see the definition of F^η in Definition 4.2). The states of $\mathcal{A}^\eta$ are sets of formulae Γ that respect logical deduction (we call them *Hintikka sets*, see below for a definition). A state Γ acts as a specification: any point (v, i) that is processed by Γ (i.e. $\Gamma = \rho_i(v)$ in some accepting run ρ) must satisfy all the formulae in Γ.[16] Spatial transitions (δ_s) are essentially obtained by dropping the modal operators $\Diamond$ and $\Box$, and temporal transitions (δ_t) by dropping the temporal operators $\bigcirc$ and $\ominus$.

From now on, we fix a formula $\eta \in \mathtt{TG}\mu$. We call *Hintikka set* a set $\Gamma \subset \mathtt{TG}\mu$ that "respects logical deduction", in the following sense: for all formulae φ, ψ,

- If $\varphi \vee \psi \in \Gamma$, then either $\varphi \in \Gamma$ or $\psi \in \Gamma$;

[15] Indeed, $\lambda : Q \to \mathcal{P}(\mathcal{P}(\mathsf{AP}))$ can equivalently by seen as a relation $\lambda \subset Q \times \mathcal{P}(\mathsf{AP})$, which is the same typing as $\delta_s \subset Q \times \mathcal{P}(Q)$, except that $\mathcal{P}(Q)$ is replaced by $\mathcal{P}(\mathsf{AP})$.

[16] This is actually exactly what Claim 4.4 in the proof of Theorem 4.3 says.

– If $\varphi \wedge \psi \in \Gamma$, then $\varphi \in \Gamma$ and $\psi \in \Gamma$;

– If $\mathtt{rec}X.\varphi \in \Gamma$, then $\varphi[\mathtt{rec}X.\varphi/X] \in \Gamma$.

Example 4.1
For all pointed QTGs $(\mathfrak{L}, v, i)$, the set

$$\Gamma = \{\varphi \in \mathtt{TG}\mu \mid \mathfrak{L}, v, i \models \varphi\}$$

is a Hintikka set.

Definition 4.2 (Automaton of η)

The *automaton of* η is the automaton $\mathcal{A}^\eta = (Q^\eta, \delta_s^\eta, \delta_t^\eta, \lambda^\eta, I^\eta, F^\eta)$, where:

- $Q^\eta = \{\Gamma \subset \mathrm{cl}(\eta) \mid \Gamma \text{ is a Hintikka set}\}$;
- For $\Gamma \in Q^\eta$ and $N \subset Q^\eta$,

$$(\Gamma, N) \in \delta_s^\eta \quad \text{iff} \quad \forall \varphi \in \mathtt{TG}\mu, \begin{cases} \Diamond\varphi \in \Gamma \Rightarrow \varphi \in \bigcup N \\ \Box\varphi \in \Gamma \Rightarrow \varphi \in \bigcap N; \end{cases}$$

- For $\Gamma, \Delta \in Q^\eta$,

$$(\Gamma, \Delta) \in \delta_t^\eta \quad \text{iff} \quad \forall \varphi \in \mathtt{TG}\mu, \begin{cases} \bigcirc\varphi \in \Gamma \Rightarrow \varphi \in \Delta \\ \ominus\varphi \in \Gamma \Rightarrow \varphi \in \Delta; \end{cases}$$

- For $\Gamma \in Q^\eta$,

$$\lambda^\eta(\Gamma) = \left\{ a \in \mathcal{P}(\mathsf{AP}) \mid \forall p \in \mathsf{AP}, \begin{array}{c} p \in \Gamma \Rightarrow p \in a \\ \neg p \in \Gamma \Rightarrow p \notin a \end{array} \right\};$$

- $I^\eta = \{\Gamma \in Q^\eta \mid \eta \in \Gamma\}$;
- $F^\eta = \{\Gamma \in Q^\eta \mid \Gamma \text{ contains no formula of the form } \bigcirc\varphi\}$.

Theorem 4.3 (Correctness of the construction)

We have

$$\mathrm{L}(\mathcal{A}^\eta) = \{(\mathfrak{L}, v_0, 0) \text{ pointed QTG} \mid \mathfrak{L}, v_0, 0 \models \eta\}.$$

Proof. Let $(\mathfrak{L}, v_0, 0)$ be a pointed QTG; we write $n = \mathrm{len}(\mathfrak{L})$. We seek to prove that

$$(\mathfrak{L}, v_0, 0) \in \mathrm{L}(\mathcal{A}^\eta) \quad \Longleftrightarrow \quad \mathfrak{L}, v_0, 0 \models \eta.$$

$\boxed{\Leftarrow}$ Suppose $\mathfrak{L}, v_0, 0 \models \eta$. For all $i \in [n]$ and $v \in V_i^{\mathfrak{L}}$, we define

$$\rho_i(v) = \{\varphi \in \mathrm{cl}(\eta) \mid \mathfrak{L}, v, i \models \varphi\} \in Q^\eta.$$

It follows immediately from the definitions that $\rho = (\rho_i)_{i\in[n]}$ is an accepting run of $(\mathfrak{L}, v_0, 0)$ in $\mathcal{A}^\eta$; hence, $(\mathfrak{L}, v_0, 0) \in \mathrm{L}(\mathcal{A}^\eta)$.

$\boxed{\Rightarrow}$ Suppose $(\mathfrak{L}, v_0, 0) \in \mathrm{L}(\mathcal{A}^\eta)$, and let ρ be an accepting run of $(\mathfrak{L}, v_0, 0)$ in $\mathcal{A}^\eta$.

> **Claim 4.4**
> For all $i \in [n]$ and $v \in V_i^{\mathfrak{L}}$, we have $\mathfrak{L}, v, i \models \bigwedge \rho_i(v)$.

Proof of the claim. We prove by an easy induction on the pair (φ, i) (Theorem 2.6) that for all $\varphi \in \mathtt{TG}\mu$, $i \in [n]$ and $v \in V_i^{\mathfrak{L}}$, if $\varphi \in \rho_i(v)$ then $\mathfrak{L}, v, i \models \varphi$. This statement directly gives Claim 4.4. ◀

In particular for $i = 0$ and $v = v_0$, we have

$$\mathfrak{L}, v_0, 0 \models \bigwedge \rho_0(v_0).$$

Since ρ is an accepting run, $\rho_0(v_0) \in I^\eta$, i.e. $\eta \in \rho_0(v_0)$. Therefore, $\mathfrak{L}, v_0, 0 \models \eta$, as desired. ◀

The principal theorems of the document (Theorem 3.5, Theorem 4.3, and then Theorem 1.7 and Proposition 2.13) now combine harmoniously to yield the small model property, which we only need to pluck like an apple fallen from the tree.

Corollary 4.5 (Small model property of $\mathtt{TG}\mu$)

Let $\eta \in \mathtt{TG}\mu$. If η is satisfiable, then:

1. There exists a pointed QTG $(\mathfrak{L}, v_0, 0)$ such that $\mathfrak{L}, v_0, 0 \models \eta$, $\mathrm{len}(\mathfrak{L}) \leq 2^{2^{|\eta|}}$ and $\mathrm{surf}(\mathfrak{L}) \leq 2^{|\eta|}$;
2. There exists a pointed TG $(\mathfrak{G}, v_0, 0)$ such that $\mathfrak{G}, v_0, 0 \models \eta$, $\mathrm{len}(\mathfrak{G}) \leq 2^{2^{|\eta|}}$ and $\mathrm{surf}(\mathfrak{G}) \leq 2^{|\eta|} \cdot 2^{2^{|\eta|}}$.

Proof.

$\boxed{1.}$ Suppose that η is satisfiable. By Theorem 4.3, $\mathrm{L}(\mathcal{A}^\eta) \neq \varnothing$, so by Theorem 3.5, there exists $(\mathfrak{L}, v_0, 0) \in \mathrm{L}(\mathcal{A}^\eta)$ (i.e. $\mathfrak{L}, v_0, 0 \models \eta$, again by Theorem 4.3) such that

$$\mathrm{len}(\mathfrak{L}) \leq 2^{|Q^\eta|} \leq 2^{2^{|\eta|}}$$
$$\mathrm{surf}(\mathfrak{L}) \leq |Q^\eta| \leq 2^{|\eta|},$$

the inequalities coming from the fact that $Q^\eta \subset \mathcal{P}(\mathrm{cl}(\eta))$ (so $|Q^\eta| \leq 2^{|\mathrm{cl}(\eta)|} = 2^{|\eta|}$).

2. Let $(\mathfrak{L}, v_0, 0)$ be given by Point 1. We apply Theorem 1.7 to $\mathfrak{L}$, which gives a TG $\mathfrak{G}$ such that $\mathfrak{L} \preceq \mathfrak{G}$ and

$$\begin{aligned} \mathrm{len}(\mathfrak{G}) &= \mathrm{len}(\mathfrak{L}) \leq 2^{2^{|\eta|}} \\ \mathrm{surf}(\mathfrak{G}) &\leq \mathrm{len}(\mathfrak{L}) \cdot \mathrm{surf}(\mathfrak{L}) \leq 2^{2^{|\eta|}} \cdot 2^{|\eta|}. \end{aligned}$$

Since $\mathfrak{L} \preceq \mathfrak{G}$, there is[a] $(w_0, 0) \in \mathrm{Pts}(\mathfrak{G})$ such that $(\mathfrak{L}, v_0, 0) \rightleftarrows (\mathfrak{G}, w_0, 0)$. Using Proposition 2.13 and the fact that $\mathfrak{L}, v_0, 0 \models \eta$, we obtain that $\mathfrak{G}, w_0, 0 \models \eta$, as desired. ◀

[a] Technical note: The definition of $\preceq$ only gives an arbitrary point (w_0, i) of $\mathfrak{G}$ (whereas we claim here that $i = 0$). It is not hard to see from the proof of Theorem 1.7 that each point of $\mathfrak{L}$ can actually be matched to a point of $\mathfrak{G}$ *that is at the same timestamp*.

Since $\mathtt{LTL}\times\mathtt{K}$ can be embedded into $\mathtt{TG}\mu$ in a way that preserves the size of formulae (Lemma 2.11), we obtain the same small model property for $\mathtt{LTL}\times\mathtt{K}$ (doubly-exponential in length and singly-exponential in surface for QTGs, and doubly-exponential for both length and surface for TGs).

The proof of the small model property for TGs (Corollary 4.5 Point 2.) can be interpreted as a procedure to compress a given TG model $\mathfrak{G}$ of a satisfiable formula η:

1. Using Theorem 4.3, turn η into an equivalent spatio-temporal automaton $\mathcal{A}^\eta$ ($\mathfrak{G}$ is therefore accepted by $\mathcal{A}^\eta$);
2. Using Theorem 3.5, compress $\mathfrak{G}$ into a QTG $\mathfrak{L}$ that is also accepted by $\mathcal{A}^\eta$ and therefore satisfies η (this step of the procedure can be interpreted as compressing $\mathfrak{G}$ "so much" that it is not exactly a TG anymore, but only a QTG);
3. Using Theorem 1.7, slightly decompress $\mathfrak{L}$ into an equivalent TG $\mathfrak{G}'$, which also satisfies η.

What we proved is that the result $\mathfrak{G}'$ of this compression procedure always has a length and a surface at most doubly-exponential in $|\eta|$.

4.2. Consequences on expressiveness

Spatio-temporal automata, just as formulae of our spatio-temporal logics, are devices that take a pointed QTG as input and return "true" or "false". We can therefore compare them with the logics in terms of expressiveness. Let $\mathtt{Aut}$ be the class of spatio-temporal automata; Theorem 4.3 says that $\mathtt{TG}\mu \sqsubseteq \mathtt{Aut}$ (i.e. for any formula $\eta \in \mathtt{TG}\mu$, there exists an automaton $\mathcal{A}^\eta$ that recognises the same pointed QTGs as η).

We give below an example of a property that automata *cannot* express.

Proposition 4.6 (Automata cannot express spatial reachability)

There is no spatio-temporal automaton $\mathcal{A}$ that recognises the class of pointed QTGs

$(\mathfrak{L}, v_0, 0)$ such that in the first graph $G_0^{\mathfrak{L}}$, there exists a node reachable from v_0 satisfying p.

Proof. Suppose that such an automaton $\mathcal{A} = (Q, \delta_s, \delta_t, \lambda, I, F)$ exists. We are going to show that $\mathcal{A}$ must have an infinite number of states, which will result in a contradiction.

Let $m \in \mathbb{N}_1$. Consider the TG $\mathfrak{G}_m$ of length 1, whose only graph G is a path of length m with its last node satisfying p. Formally, we define $G = (V, E, L)$, where $V = \{v_0, \dots, v_m\}$, $E = \{(v_k, v_{k+1}),\ 0 \leq k < m\}$, $L(v_k) = \varnothing$ for $k < m$ and $L(v_m) = \{p\}$.

Clearly, $(\mathfrak{G}_m, v_0, 0) \in \mathrm{L}(\mathcal{A})$ (v_m is reachable from v_0, and satisfies p), so there exists an accepting run $\rho = \rho_0 : V \to Q$ of $(\mathfrak{G}_m, v_0, 0)$ in $\mathcal{A}$. We claim that on this run, all the nodes must be matched to different states:

> **Claim 4.7**
> The function ρ is injective.

Proof of the claim. Suppose by contradiction that there are $0 \leq p < q \leq m$ such that $\rho(v_p) = \rho(v_q)$. Consider the TG $\mathfrak{G}'$ of length 1, whose only graph G' is the same path as for $\mathfrak{G}_m$, but instead of going to v_m, it loops on itself between v_p and v_q. Formally, $G' = (V', E', L')$, where $V' = \{v_0, \dots, v_{q-1}\}$, $E' = \{(v_k, v_{k+1}),\ 0 \leq k \leq q-2\} \cup \{(v_{q-1}, v_p)\}$ and for all $0 \leq k \leq q-1$, $L'(v_k) = \varnothing$.

Then the run $\tau : V' \to Q$ defined for all $0 \leq k \leq q-1$ by $\tau(v_k) = \rho(v_k)$ is an accepting run of $(\mathfrak{G}', v_0, 0)$ in $\mathcal{A}$; therefore, $(\mathfrak{G}', v_0, 0) \in \mathrm{L}(\mathcal{A})$. However, no node satisfying p is reachable from v_0 in G', hence a contradiction. We conclude that ρ is injective. ◀

As a consequence, $|Q| \geq m+1$. That being true for all $m \in \mathbb{N}_1$, we conclude that $|Q| = \infty$, a contradiction. Thus, the automaton $\mathcal{A}$ cannot exist. ◀

We list below some consequences of Proposition 4.6.

Consequence 1. Languages recognised by spatio-temporal automata are not closed under complement, as shown by Example 3.4 and Proposition 4.6. It seems that spatio-temporal automata can explore a single graph to an arbitrary depth to verify safeness properties (Example 3.4), but not reachability properties (Proposition 4.6).

Consequence 2. With the help of a small trick, we can also deduce the strict comparison $\boxed{\texttt{TG}\mu \sqsubset \texttt{Aut}}$; specifically, that there is no $\texttt{TG}\mu$ formula equivalent to the automaton of Example 3.4. Indeed, suppose that such a formula η exists. Then $\sim\eta$ accepts the pointed QTGs such as in the current graph, some node that does not satisfy p is reach-

able.[17] Then $\mathcal{A}^\eta$ also accepts this language, which contradicts Proposition 4.6.

In the Conclusion section, we provide a candidate to logically characterise the expressiveness of the class `Aut`.

4.3. The satisfiability algorithm

Let $\eta \in \mathtt{TG}\mu$. The procedure to decide the satisfiability of η consists in building $\mathcal{A}^\eta$, and then applying one of the algorithms of Section 3.2 (the nondeterministic or the deterministic one) to check for the nonemptiness of its language.

In order not to obtain a suboptimal complexity for this procedure, we use the compact representations described in Section 3.3 to store $\mathcal{A}^\eta$: for $\Gamma \in Q^\eta$, we define

$$\varepsilon_s^\eta(\Gamma) = \left[\bigwedge_{(\Diamond\varphi)\in\Gamma} \bigvee_{\substack{\Delta\in Q^\eta \\ \varphi\in\Delta}} \Delta\right] \wedge \left[\bigwedge_{(\Box\varphi)\in\Gamma} \bigwedge_{\substack{\Delta\in Q^\eta \\ \varphi\notin\Delta}} (\neg\Delta)\right] \quad \text{and} \quad \mu^\eta(\Gamma) = \bigwedge_{\substack{p\in\mathsf{AP} \\ p\in\Gamma}} p \wedge \bigwedge_{\substack{p\in\mathsf{AP} \\ (\neg p)\in\Gamma}} (\neg p).$$

It follows directly from the definitions that the induced spatial transition function of ε_s^η and the induced labeling of μ^η are indeed δ_s^η and λ^η, respectively. Moreover, ε_s^η can be stored with space $2^{\mathcal{O}(|\eta|)}$, and μ^η with space $\mathcal{O}(|\eta|^2)$. Therefore, the full automaton $\mathcal{A}^\eta$ can be stored in a space $2^{\mathcal{O}(|\eta|)}$.

Running the procedure deciding satisfiability of η requires to store $\mathcal{A}^\eta$ (which, as we just saw, needs a space $2^{\mathcal{O}(|\eta|)}$) and the space used by the algorithm to decide the nonemptiness of its language (which needs a space $\mathcal{O}(|Q^\eta|) = 2^{\mathcal{O}(|\eta|)}$). Therefore, the whole procedure runs with a (nondeterministic or deterministic, depending on the algorithm of Section 3.2 used) singly-exponential space. In any case (even if it means using Savitch's theorem, which says that ExpSpace = NExpSpace), we obtain an ExpSpace complexity.

Theorem 4.8 (Complexity of satisfiability in $\mathtt{TG}\mu$)

The problem of deciding satisfiability in $\mathtt{TG}\mu$ is in ExpSpace.

5. Matching lower bounds

In this section, we show that our small model property result (Corollary 4.5) is optimal, by providing corresponding matching lower bounds. The small model property is given in two versions, one for the class of QTGs and one for the class of TGs. We therefore also give two separate results for the lower bounds:

[17] The formula $\sim\eta \in \mathtt{TG}\mu$ is the semantic negation of η. It is obtained by replacing all the boolean and modal operators in η by their duals, and the atomic propositions by their negations (`rec` is self-dual). For all pointed QTGs $(\mathfrak{L}, v, i)$, we have $\mathfrak{L}, v, i \models \sim\eta$ iff $\mathfrak{L}, v, i \not\models \eta$.

- (QTGs) there exists satisfiable formulae $\Lambda_n \in \mathtt{LTL}\times\mathtt{K}$, the length (resp. surface) of any QTG model of which grows doubly-exponentially (resp. singly-exponentially) with $|\Lambda_n|$ (Section 5.1 and Theorem 5.1);
- (TGs) there exists satisfiable formulae $\Phi_n \in \mathtt{LTL}\times\mathtt{K}$, the length (resp. surface) of any TG model of which grows doubly-exponentially (resp. doubly-exponentially) with $|\Phi_n|$ (Section 5.2 and Theorem 5.2).

Since $\mathtt{LTL}\times\mathtt{K}$ can be embedded into $\mathtt{TG}\mu$ in a way that preserves the size of formulae (Lemma 2.11), the Λ_n and Φ_n can as well be seen as $\mathtt{TG}\mu$ formulae, so that we obtain matching lower bounds for the small model properties of both $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$ at the same time.

5.1. Lower bound for QTGs

This section aims at providing the lower bound for the class of QTGs. As a reminder, we seek to enforce models to have a *doubly-exponential length* and *singly-exponential surface* (in the size of the constructed formula). Clearly, the main challenge is for the length (indeed, reaching a singly-exponential surface is already possible in K, without the "help" of the temporal dimension[18]).

Our main ingredient to meet this challenge is *counters*, which are a standard construction in LTL. To make the exposition easier to follow, we first present counters in LTL, and then show how to use them in the spatio-temporal setting to enforce lengthy models of $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$.

Counters in LTL. Let us consider LTL interpreted on finite traces, that is, finite words over the alphabet $\Sigma = \mathcal{P}(\mathsf{AP})$. Let $b_0, \dots, b_{n-1} \in \mathsf{AP}$; we use those atomic propositions to represent the binary decomposition of a number in $[\![0, 2^n - 1]\!]$ (b_k gives the value of the bit of weight k of the number). More precisely, to a letter $a \in \Sigma$ we associate the number

$$\overline{a} = \sum_{k=0}^{n-1} \mathbb{1}_a(b_k) \cdot 2^k \in [\![0, 2^n - 1]\!]$$

(where $\mathbb{1}_a$ is the indicator function of a, i.e. $\mathbb{1}_a(b_k) = 1$ if $b_k \in a$, and $\mathbb{1}_a(b_k) = 0$ if not). As an example, $\overline{\{b_1, b_2\}} = 6$. We further associate to a trace $w = a_0 \cdots a_{p-1} \in \Sigma^p$ the sequence of integers $\overline{w} = \overline{a_0} \cdots \overline{a_{p-1}} \in [\![0, 2^n - 1]\!]^p$. As an example, if $w = (\varnothing, \{b_2\}, \{b_0, b_3\})$, then $\overline{w} = (0, 4, 9)$.

[18] The standard construction is a formula τ_n (whose size grows linearly with n), that builds a perfect binary tree (that is, a binary tree where all interior nodes have two children, and all leaves are on the same depth) of height n (which thus has $2^{n+1} - 1$ nodes).

Let $\alpha_n, \beta_n \in \mathtt{LTL}$ be defined as

$$\alpha_n = (b_0 \to \bigcirc\neg b_0) \wedge (\neg b_0 \to \bigcirc b_0) \wedge \bigwedge_{k=0}^{n-2} \underbrace{(\bigcirc b_{k+1} \leftrightarrow (b_{k+1} \leftrightarrow (b_k \to \bigcirc b_k)))}_{(***)}$$

$$\beta_n = \underbrace{\left(\bigwedge_{k=0}^{n-1} \neg b_k\right)}_{(*)} \wedge [\alpha_n \,\mathtt{U}\, \underbrace{\left(\bigwedge_{k=0}^{n-1} b_k \wedge \ominus\mathtt{ff}\right)}_{(**)}].$$

Then β_n acts like an n-bit counter, in the sense that its only model is the word $w \in \Sigma^{2^n}$ such that $\overline{w} = (0, 1, \ldots, 2^n - 1)$. It should also be noted that the size of β_n is linear in n.

Let us explain how β_n works. The subformulae $(*)$ and $(**)$ specify the extremal conditions, i.e. that w should start with $\varnothing$ and end with $\{b_0, \ldots, b_{n-1}\}$ (or, equivalently, that $\overline{w}$ should start with 0 and end with $2^n - 1$). The subformula α_n is the core of the machinery of the counter. It uses the fact that the value that b_{k+1} should have at timestamp $i+1$ (that we may abbreviate as $b_{k+1}(i+1)$) only depends on $b_k(i)$, $b_k(i+1)$ and $b_{k+1}(i)$ (the abbreviations work on the same principle as $b_{k+1}(i+1)$). As examples:

- if $b_k(i) = 0$, $b_k(i+1) = 1$ and $b_{k+1}(i) = 0$, then $b_{k+1}(i+1) = 0$;
- if $b_k(i) = 1$, $b_k(i+1) = 0$ and $b_{k+1}(i) = 0$, then $b_{k+1}(i+1) = 1$.

The general form is given in a succint way by the subformula $(***)$.

For our purposes, we will actually use another version of the LTL counter, that we may call the *periodic counter*. As in the basic version above, the counter starts at 0 and increases, but instead of stopping at $2^n - 1$, it stops at another value $P \in [\![0, 2^n - 1]\!]$, whose binary decomposition is given by the propositional constants $p_0, \ldots, p_{n-1} \in \{\mathtt{tt}, \mathtt{ff}\}$. Then, it has the choice either to end the word, or to restart from 0. Formally, we define

$$\pi_n = \left(\bigwedge_{k=0}^{n-1} \neg b_k\right) \wedge \mathtt{G}\left[\left(\bigwedge_{k=0}^{n-1} (b_k \leftrightarrow p_k) \to \ominus \bigwedge_{k=0}^{n-1} \neg b_i\right) \wedge \left(\neg \bigwedge_{k=0}^{n-1} (b_k \leftrightarrow p_k) \to \alpha_n\right)\right].$$

Then the models of π_n are exactly the words w such that $\overline{w} \in (0, \ldots, P)^+$ (where $-^+$ is the Kleene plus). They are periodic with period $P+1$. *In particular, the length of these models is always a (nonzero) multiple of $P+1$.*

Lengthy models of LTL×K. We start by giving a high-level description of the construction. The basic counter formula β_n is enough alone to obtain a singly-exponential lower bound on the length of QTG models. To reach the doubly-exponential lower bound, we start multiple (specifically, 2^n) periodic counters from different nodes of the first graph, whose periods $P+1$ cover all the values in $\{1, \ldots, 2^n\}$. Since QTGs are synchronous[19], the length of the model must be a common multiple of $1, \ldots, 2^n$, and

[19] Which means that all the temporal traces (and in that case, all the counters) must end at the same timestamp (namely, the last one), see the Introduction section.

therefore $\geq \operatorname{lcm}(1, \dots, 2^n)$, which, according to number theorists, grows doubly exponentially with n.

Formally, we define $\mathsf{AP} = \{b_k,\ k \in \mathbb{N}_0\} \cup \{p_k,\ k \in \mathbb{N}_0\}$.[20] Let $n \in \mathbb{N}_0$. Let $\tau_n \in \mathtt{K}$ be a formula (whose size grows linearly with n) that builds a perfect binary tree[21] of height n, and whose leaves satisfy all the combinations of the atomic propositions $p_0, \dots, p_{n-1}$. As an example, we may define

$$\tau_2 = \Diamond\neg p_0 \wedge \Diamond p_0 \wedge \Box\left[(\neg p_0 \to \Box\neg p_0) \wedge (p_0 \to \Box p_0) \wedge \Diamond\neg p_1 \wedge \Diamond p_1\right].$$

The built tree has one leaf that contains neither p_0 nor p_1 in its label, one that contains p_0 and not p_1, one that contains p_1 and not p_0 and one that contains both p_0 and p_1.

The 2^n leaves of the tree are the nodes from which we will start the periodic counters. Given $\psi \in \mathtt{LTL}{\times}\mathtt{K}$, we let $\tau_n[\psi] \in \mathtt{LTL}{\times}\mathtt{K}$ be as τ_n, except that we further demand that all the leaves of the tree satisfy ψ. Again, as an example, we may define

$$\tau_2[\psi] = \Diamond\neg p_0 \wedge \Diamond p_0 \wedge \Box\left[(\neg p_0 \to \Box\neg p_0) \wedge (p_0 \to \Box p_0) \wedge \Diamond\neg p_1 \wedge \Diamond p_1 \wedge \Box\psi\right].$$

We then define

$$\Lambda_n = \tau_n\left[\bigwedge_{k=0}^{n-1}((s_k \to \mathtt{G}s_k) \wedge (\neg s_k \to \mathtt{G}\neg s_k)) \wedge \pi_n\right].$$

The Λ_n satisfy the requirements of Theorem 5.1 below, and are therefore matching lower bounds for the class of QTGs.

Theorem 5.1 (Lower bound - QTGs)

There exists a sequence of formulae $(\Lambda_n)_{n\in\mathbb{N}_0} \in (\mathtt{LTL}{\times}\mathtt{K})^{\mathbb{N}_0}$ such that:

- $|\Lambda_n| \underset{n\to\infty}{=} \mathcal{O}(n)$;
- For all $n \in \mathbb{N}_0$, Λ_n is satisfiable;
- For all $n \in \mathbb{N}_0$, any (pointed) QTG that satisfies Λ_n has a length $\geq \ell_n$ and a surface $\geq s_n$, where

$$\ell_n \underset{n\to\infty}{=} \Omega\left((\sqrt{\mathrm{e}})^{2^n}\right) \quad \text{and} \quad s_n = 2^n.$$

Proof. Let us check that the Λ_n satisfy the three points in the statement of the theorem.

Size The size of τ_n and $\psi_n = \bigwedge_{k=0}^{n-1}((s_k \to \mathtt{G}s_k) \wedge (\neg s_k \to \mathtt{G}\neg s_k)) \wedge \pi_n$ both grow

[20] Here, the p_k are atomic propositions, whereas in the previous paragraph, they were propositional constants (that is, elements of $\{\mathtt{tt}, \mathtt{ff}\}$). This difference will not change how they function in formulae (because their value will be fixed anyway).

[21] That is, a binary tree where all interior nodes have two children, and all leaves are on the same depth.

linearly with n. $\Lambda_n = \tau_n[\psi_n]$ is obtained from τ_n by adding "$\wedge\ \Box\psi_n$" somewhere inside it; therefore, its size also grows linearly with n.

Satisfiability Let T_n be a graph that satisfies τ_n (T_n is thus a perfect binary tree of height n). Then Λ_n has a TG model of length $\mathrm{lcm}(1,\dots,2^n)$ on the node set T_n.

Lower bound Let $(\mathfrak{L}, v_0, 0)$ be a pointed QTG such that $\mathfrak{L}, v_0, 0 \models \Lambda_n$. Then $G_0^{\mathfrak{L}}, v_0 \models_{\mathtt{K}} \tau_n$, so $G_0^{\mathfrak{L}}$ must contain a perfect binary tree of height n rooted in v_0. In particular,

$$\mathrm{surf}(\mathfrak{L}) \geq |V_0^{\mathfrak{L}}| \geq 2^{n+1} - 1 \geq 2^n = s_n.$$

For the length: because of the counters, $\mathrm{len}(\mathfrak{L})$ must be a common multiple of $1,\dots,2^n$, so it is a (nonzero) multiple of $\ell_n = \mathrm{lcm}(1,\dots,2^n)$. In particular, we have $\mathrm{len}(\mathfrak{L}) \geq \ell_n$. Moreover, it is standard knowledge in number theory that

$$\mathrm{lcm}(1,\dots,N) \underset{N\to\infty}{=} \Omega\left((\sqrt{\mathrm{e}})^N\right).$$

(More precisely: the Chebyshev function ψ, which satisfies $\mathrm{e}^{\psi(N)} = \mathrm{lcm}(1,\dots,N)$ for all integers N, has the asymptotic behaviour $\psi(x) \underset{x\to\infty}{\sim} x$ – a statement equivalent to the prime number theorem, see e.g. [Cha68] p65. In particular, for N large enough, we have $\psi(N) \geq N/2$, so $\mathrm{lcm}(1,\dots,N) \geq (\sqrt{\mathrm{e}})^N$.) Hence,

$$\ell_n = \mathrm{lcm}(1,\dots,2^n) \underset{n\to\infty}{=} \Omega\left((\sqrt{\mathrm{e}})^{2^n}\right),$$

as claimed. ◀

5.2. Lower bound for TGs

Let us now construct the lower bound for the class of TGs. The formulae Λ_n already satisfy the requirements regarding the length. To obtain a doubly-exponential surface for TGs, we add to Λ_n a small artefact: the formula σ of Example 2.3.

Specifically, the formulae $\Phi_n = (\Diamond\Lambda_n) \wedge (\Diamond\sigma)$ satisfy the requirements of Theorem 5.2 below.

Theorem 5.2 (Lower bound - TGs)

There exists a sequence of formulae $(\Phi_n)_{n\in\mathbb{N}_0} \in (\mathtt{LTL}\times\mathtt{K})^{\mathbb{N}_0}$ such that:

- $|\Phi_n| \underset{n\to\infty}{=} \mathcal{O}(n)$;
- For all $n \in \mathbb{N}_0$, Φ_n is satisfiable;
- For all $n \in \mathbb{N}_0$, any (pointed) TG that satisfies Φ_n has a length $\geq \ell_n$ and a surface $\geq s_n$, where

$$\ell_n \underset{n\to\infty}{=} \Omega\left((\sqrt{\mathrm{e}})^{2^n}\right) \quad \text{and} \quad s_n \underset{n\to\infty}{=} \Omega\left((\sqrt{\mathrm{e}})^{2^n}\right).$$

Proof. Again, let us check that the Φ_n satisfy the three points in the statement of the theorem.

Size This point is clear since $|\Lambda_n|$ grows linearly with n (as seen in Theorem 5.1) and σ does not depend on n.

Satisfiability The satisfiability of Φ_n boils down to the following claim, together with the facts that Λ_n is satisfiable and σ has models of any length.

> **Claim 5.3**
> Let $\varphi, \psi \in \mathtt{LTL}\times\mathtt{K}$ that both have a model of length q. Then $(\Diamond\varphi) \wedge (\Diamond\psi)$ also has a model of length q.

(The $\Diamond$ are here so that φ and ψ do not interfere. The result is not true without them: take e.g. $\varphi = p \in \mathsf{AP}$ and $\psi = \neg p$.)

Lower bound Define $\ell_n = s_n = \mathrm{lcm}(1, \dots, 2^n)$. It has already be seen in Theorem 5.1 that this quantity is $\underset{n\to\infty}{=} \Omega\left((\sqrt{\mathrm{e}})^{2^n}\right)$.

Let $(\mathfrak{G}, v_0, 0)$ be a pointed TG such that $\mathfrak{G}, v_0, 0 \models \Phi_n$. Then there exists two nodes $w, z \in V_0^{\mathfrak{G}}$ such that $\mathfrak{G}, w, 0 \models \Lambda_n$ and $\mathfrak{G}, z, 0 \models \sigma$ (w and z are neighbours of v_0).

By Theorem 5.1, we deduce that $\mathrm{len}(\mathfrak{G}) \geq \ell_n$. We also deduce from the result of Example 2.3 that

$$\mathrm{surf}(\mathfrak{G}) \geq \mathrm{len}(\mathfrak{G}) \geq \ell_n = s_n,$$

as wanted. ◀

Conclusion

We studied two logics, $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$, interpreted on QTGs and TGs. Although the class of QTGs is (strictly) larger than the class of TGs, the two are equivalent at the proof-theoretical level: a formula in $\mathtt{LTL}\times\mathtt{K}$ or $\mathtt{TG}\mu$ (or actually, any spatio-temporal-bisimulation-invariant logic) is satisfiable in a QTG iff it is in a TG.

We proved that both logics admit a small model property. Its statement is different depending on the class of models: while it is doubly-exponential in the temporal dimension and singly-exponential in the spatial dimension for the class of QTGs, it becomes doubly-exponential in both the temporal and spatial dimension for the class of TGs. Hence, QTGs are more concise models than TGs, in the spatial dimension. We provided formulae that match these upper bounds, thus showing that they are optimal.

We also gave an algorithm in ExpSpace to decide satisfiability of a formula in $\mathtt{LTL}\times\mathtt{K}$ or $\mathtt{TG}\mu$. It is based on an exponential translation of the input formula into a spatio-temporal automaton, followed by a PSpace (in the size of the automaton) procedure to decide the nonemptiness of its language. The obtained ExpSpace complexity is actually much larger than the one of the satisfiability problem of the corresponding one-dimensional logics $\mathtt{LTL}$, $\mathtt{LT}\mu$ and $\mathtt{K}$ (which is PSpace-complete) as well as when

interpreting $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$ on the broader class of asynchronous QTGs, as is done in the TIME paper (it is PSPACE-complete for $\mathtt{LTL}\times\mathtt{K}$ and EXPTIME-complete for $\mathtt{TG}\mu$). We do not have lower bounds for the complexity of satisfiability in $\mathtt{LTL}\times\mathtt{K}$ or $\mathtt{TG}\mu$ interpreted on (Q)TGs (apart from the trivial PSPACE-hardness given by the embedding of $\mathtt{LTL}$, or $\mathtt{K}$, into them).

Future work. We finish by giving some lines of research for future work.

The complexity of satisfiability. The most natural way to extend our work would be to find the actual computational complexity of the satisfiability problem in $\mathtt{LTL}\times\mathtt{K}$ and $\mathtt{TG}\mu$. Is it EXPSPACE-complete, or does there exist a better algorithm than the one presented here? Since we were able to obtain matching lower bounds for the small model property, we tend to think the former is true, but it is nothing more than an intuition. Encoding problems into (Q)TGs has proven to be challenging, notably because of the lack of the commutativity axioms $\bigcirc\Diamond p \leftrightarrow \Diamond\bigcirc p$ cherished by product logics [GKWZ03]: if we build a graph at a given timestamp i, we can talk about what each of its nodes becomes at timestamp $i+1$, but we cannot really specify properties on how they should be connected at timestamp $i+1$ depending on how they were connected at timestamp i.

The expressiveness of spatio-temporal automata. Another line of work would be to logically characterise the expressiveness of the class $\mathtt{Aut}$ of spatio-temporal automata. We saw in Section 4.2 that $\mathtt{TG}\mu \sqsubset \mathtt{Aut}$. Specifically, automata can explore a single graph at a given timestamp to an arbitrary depth to check for *safety* properties, but not *reachability* ones (unlike $\mathtt{TG}\mu$ which cannot explore a single graph to an arbitrary depth under any circumstance).

Before doing any expressiveness comparison, we should however slightly modify our definition of spatio-temporal automata. Indeed, for the sake of simplicity, we required in Definition 3.2 for a run to be defined on *every point of a QTG*. This enables automata to globally talk about graphs, and therefore express non-bisimulation-invariant properties. As an example, the automaton $(\{*\}, \delta_s, \delta_t, \lambda, \{*\}, \{*\})$, where

$$\begin{aligned}\delta_s &= \{(*, \varnothing)\}\\ \delta_t &= \{(*, *)\}\\ \lambda(*) &= \mathcal{P}(\mathsf{AP})\end{aligned}$$

recognises the class of pointed QTGs with no edges (that is, $E_i = \varnothing$ for all timestamps i). This issue can easily be fixed by allowing, in Definition 3.2, the functions ρ_i to be partial, and only defined on the points reachable from $(v_0, 0)$ using (spatial) edges or temporal successor functions.

That being said, a natural candidate to match the expressiveness of $\mathtt{Aut}$ would be the

logic defined by the syntax

$$\begin{aligned}\varphi ::= &\mid p \mid \neg p \mid X && (p \in \mathsf{AP},\ X \in \mathsf{Var})\\ &\mid \varphi \vee \varphi \mid \varphi \wedge \varphi\\ &\mid \bigcirc\varphi \mid \ominus\varphi\\ &\mid \Diamond\varphi \mid \Box\varphi\\ &\mid \nu X.\varphi && (X \in \mathsf{Var}),\end{aligned}$$

where we *do not* require formulae to be temporarily-guarded, and where ν is interpreted as a greatest fixpoint.

References

[ALS26] Eric Alsmann, Martin Lange, and Igor Semezies. The Satisfiability Problem of Temporal-Spatial Logics over Quasi-Temporal Graphs. In AndreA Orlandini and Sophie Pinchinat, editors, *33rd International Symposium on Temporal Representation and Reasoning (TIME 2026)*, volume 146 of *Open Access Series in Informatics (OASIcs)*, pages 8:1–8:16, Dagstuhl, Germany, 2026. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

[BB89] B. Banieqbal and H. Barringer. Temporal logic with fixed points. In *Proc. Coll. on Temporal Logic in Specification*, volume 398 of *LNCS*, pages 62–73. Springer, 1989.

[BFL15] Florian Bruse, Oliver Friedmann, and Martin Lange. On guarded transformation in the modal μ-calculus. *Logic Journal of the IGPL*, 23(2):194–216, 04 2015.

[BKM+20] Pablo Barceló, Egor V. Kostylev, Mikaël Monet, Jorge Pérez, Juan L. Reutter, and Juan Pablo Silva. The logical expressiveness of graph neural networks. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020.

[BLMT24] Michael Benedikt, Chia-Hsuan Lu, Boris Motik, and Tony Tan. Decidability of Graph Neural Networks via Logical Characterizations. In Karl Bringmann, Martin Grohe, Gabriele Puppis, and Ola Svensson, editors, *51st International Colloquium on Automata, Languages, and Programming, ICALP 2024, Tallinn, Estonia, July 8-12, 2024*, volume 297 of *LIPIcs*, pages 127:1–127:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.

[Cha68] Komaravolu Chandrasekharan. *Introduction to analytic number theory*. Die Grundlehren der mathematischen Wissenschaften in Einzeldarstellungen, Bd. 148. Springer-Verlag, 1968.

[DGL16] Stéphane Demri, Valentin Goranko, and Martin Lange. *Temporal Logics in Computer Science: Finite-State Systems*. Cambridge Tracts in Theoretical Computer Science. Cambridge University Press, 2016.

[GKWZ03] Dov M Gabbay, Agi Kurucz, Frank Wolter, and Michael Zakharyaschev. *Many-dimensional modal logics: theory and applications*. Studies in Logic and the Foundations of Mathematics. Elsevier, 2003.

[GV13] G. De Giacomo and M. Y. Vardi. Linear Temporal Logic and Linear Dynamic Logic on Finite Traces. In *Proc. 23rd Int. Joint Conf. on A.I., IJCAI'13*, pages 854–860. IJCAI/AAAI, 2013.

[Kna28] Bronisław Knaster. Un théorème sur les fonctions d'ensembles. *Ann. Soc. Polon. Math.*, 6:133–134, 1928.

[Koz83] Dexter Kozen. Results on the propositional μ-calculus. *Theoretical Computer Science*, 27(3):333–354, 1983. Special Issue Ninth International Colloquium on Automata, Languages and Programming (ICALP) Aarhus, Summer 1982.

[LDTC16] Y. Liu, Z. Duan, C. Tian, and B. Cui. Satisfiability of Linear Time Mu-Calculus on Finite Traces. In *Proc. 22nd Int. Conf. on Computing and Combinatorics, COCOON'16*, volume 9797 of *LNCS*, pages 611–622. Springer, 2016.

[LLS+23] Antonio Longa, Veronica Lachi, Gabriele Santin, Monica Bianchini, Bruno Lepri, Pietro Lio, Franco Scarselli, and Andrea Passerini. Graph Neural Networks for Temporal Graphs: State of the Art, Open Challenges, and Opportunities. *Trans. Mach. Learn. Res.*, 2023, 2023.

[LWZ08] Carsten Lutz, Frank Wolter, and Michael Zakharyashev. Temporal Description Logics: A Survey. In *Proceedings of the International Workshop on Temporal Representation and Reasoning*, pages 3 – 14, 07 2008.

[Mat02] Radu Mateescu. Local Model-Checking of Modal Mu-Calculus on Acyclic Labeled Transition Systems. In Joost-Pieter Katoen and Perdita Stevens, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 281–295, Berlin, Heidelberg, 2002. Springer Berlin Heidelberg.

[MV97] M. Marx and Y. Venema. *Multi-Dimensional Modal Logic*, pages 1–9. Springer, Dordrecht, 1997.

[NSST24] Pierre Nunn, Marco Sälzer, François Schwarzentruber, and Nicolas Troquard. A Logic for Reasoning about Aggregate-Combine Graph Neural Networks. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI 2024, Jeju, South Korea, August 3-9, 2024*, pages 3532–3540. ijcai.org, 2024.

[PDC+20] Aldo Pareja, Giacomo Domeniconi, Jie Chen, Tengfei Ma, Toyotaro Suzumura, Hiroki Kanezashi, Tim Kaler, Tao B. Schardl, and Charles E. Leiserson. EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, pages 5363–5370. AAAI Press, 2020.

[SC85] A. P. Sistla and E. M. Clarke. The complexity of propositional linear temporal logics. *Journal of the ACM*, 32(3):733–749, 1985.

[SWL25] Marco Sälzer, Przemysław Wałęga, and Martin Lange. The Logical Expressiveness of Temporal GNNs via Two-Dimensional Product Logics. In Danielle Belgrave, Cheng Zhang, Laura N. Montoya, Hsuan-Tien Lin, Razvan Pascanu, Piotr Koniusz, Marzyeh Ghassemi, Nancy Chen, Iván Vladimir Meza Ruíz, and Arturo Loaiza-Bonilla, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*, 2025.

[Tar55] Alfred Tarski. A lattice-theoretical fixpoint theorem and its applications. *Pacific Journal of Mathematics*, 5:285–309, 1955.

[Var88] M. Y. Vardi. A temporal fixpoint calculus. In *Proc. Conf. on Principles of Programming Languages, POPL'88*, pages 250–259. ACM, 1988.

[VW86] Moshe Y Vardi and Pierre Wolper. An automata-theoretic approach to automatic program verification. In *Proceedings of the First Symposium on Logic in Computer Science*. Institute of Electrical and Electronics Engineers (IEEE), 1986.

A. From QTGs to TGs: the proof

The aim of this section is to give a complete proof of Theorem 1.7. We divide it into two steps: first, the construction of $\mathfrak{J}$, whose temporal successor functions are injective (Section A.1), and second, the construction of the actual TG $\mathfrak{G}$ (Section A.2).

A.1. Step 1: building $\mathfrak{J}$

We first show how, given a QTG $\mathfrak{L}$ and a timestamp k, to make the k-th temporal successor function of $\mathfrak{L}$ injective. We call the resulting QTG the k-th *untangling of* $\mathfrak{L}$, and denote it $\mathrm{U}_k(\mathfrak{L})$.

Definition A.1 (Untangling of a QTG)

Let $\mathfrak{L}$ be a QTG of length n, and $k \in [n-1]$. The k-th *untangling of* $\mathfrak{L}$ is the QTG $\mathrm{U}_k(\mathfrak{L})$ of length n, defined by:

- For all $i \in [n] \setminus \{k+1\}$, $G_i^{\mathrm{U}_k(\mathfrak{L})} = G_i^{\mathfrak{L}}$.
- $G_{k+1}^{\mathrm{U}_k(\mathfrak{L})} = (V_{k+1}^{\mathrm{U}_k(\mathfrak{L})}, E_{k+1}, L_{k+1})$, where:
  - $V_{k+1}^{\mathrm{U}_k(\mathfrak{L})} = (V_{k+1}^{\mathfrak{L}} \setminus \mathrm{Im}(\sigma_k^{\mathfrak{L}})) \uplus V_k^{\mathfrak{L}}$. For $x \in V_{k+1}^{\mathrm{U}_k(\mathfrak{L})}$, we write
  $$x^\dagger = \begin{cases} x & \text{if } x \in V_{k+1}^{\mathfrak{L}} \setminus \mathrm{Im}(\sigma_k^{\mathfrak{L}}) \\ \sigma_k^{\mathfrak{L}}(x) & \text{if } x \in V_k^{\mathfrak{L}}. \end{cases}$$
  (Note that for all $x \in V_{k+1}^{\mathrm{U}_k(\mathfrak{L})}$, we have $x^\dagger \in V_{k+1}^{\mathfrak{L}}$.)
  - $E_{k+1} = \{(x,y) \in V_{k+1}^{\mathrm{U}_k(\mathfrak{L})} \times V_{k+1}^{\mathrm{U}_k(\mathfrak{L})} \mid (x^\dagger, y^\dagger) \in E_{k+1}^{\mathfrak{L}}\}$.
  - For $x \in V_{k+1}^{\mathrm{U}_k(\mathfrak{L})}$, $L_{k+1}(x) = L_{k+1}^{\mathfrak{L}}(x^\dagger)$.
- For all $i \in [n-1] \setminus \{k, k+1\}$, $\sigma_i^{\mathrm{U}_k(\mathfrak{L})} = \sigma_i^{\mathfrak{L}}$.
- For $v \in V_k^{\mathrm{U}_k(\mathfrak{L})} (= V_k^{\mathfrak{L}})$, $\sigma_k^{\mathrm{U}_k(\mathfrak{L})}(v) = v$.
- (If $k+1 < n-1$) For $x \in V_{k+1}^{\mathrm{U}_k(\mathfrak{L})}$, $\sigma_{k+1}^{\mathrm{U}_k(\mathfrak{L})}(x) = \sigma_{k+1}^{\mathfrak{L}}(x^\dagger)$.

The notations are very heavy, to it seems appropriate to explain what they mean. The set $V_{k+1}^{\mathrm{U}_k(\mathfrak{L})}$ (i.e. the set of nodes of $\mathrm{U}_k(\mathfrak{L})$ at timestamp $k+1$) is built from $V_{k+1}^{\mathfrak{L}}$ as follows.

- The nodes of $V_{k+1}^{\mathfrak{L}}$ that have no temporal predecessor (i.e. the nodes in the set $V_{k+1}^{\mathfrak{L}} \setminus \mathrm{Im}(\sigma_k^{\mathfrak{L}})$) are left unchanged. This corresponds to the left part of the disjoint union defining $V_{k+1}^{\mathrm{U}_k(\mathfrak{L})}$.
- For the nodes of $V_{k+1}^{\mathfrak{L}}$ that have at least one temporal predecessor, we copy them

the same number as their number of predecessors, *and actually rename each copy with the name of the corresponding predecessor*. Hence, a node $w \in V_{k+1}^{\mathfrak{L}}$ that has $s \geq 1$ predecessors $v_1, \dots, v_s \in V_k^{\mathfrak{L}}$ is replaced in $\mathrm{U}_k(\mathfrak{L})$ by these predecessors $v_1, \dots, v_s$. This corresponds to the right part of the disjoint union.

For $x \in V_{k+1}^{\mathrm{U}_k(\mathfrak{L})}$, $x^\dagger$ is the node of $V_{k+1}^{\mathfrak{L}}$ "to which x corresponds"; that is, itself if x is in the first part of the disjoint union, or the node x is predecessor of in $\mathfrak{L}$ (that is, $\sigma_k^{\mathfrak{L}}(x)$) if x is in the second part of the disjoint union.

We can observe that the k-th temporal successor function of $\mathrm{U}_k(\mathfrak{L})$ is indeed injective (see the fourth point in the definition). Moreover, we have the following:

Lemma A.2

For all QTGs $\mathfrak{L}$ and $k \in [\mathrm{len}(\mathfrak{L}) - 1]$, we have $\mathfrak{L} \preceq \mathrm{U}_k(\mathfrak{L})$.

Proof. The relation

$$\begin{aligned}\beta = \{&((v,i),(v,i)),\ i \in [\mathrm{len}(\mathfrak{L})] \setminus \{k+1\},\ v \in V_i^{\mathfrak{L}}\} \\ \cup\ \{&((x^\dagger, k+1),(x,k+1)),\ x \in V_{k+1}^{\mathrm{U}_k(\mathfrak{L})}\}\end{aligned}$$

is a bisimulation between $\mathfrak{L}$ and $\mathrm{U}_k(\mathfrak{L})$, such that every point of $\mathfrak{L}$ is connected to at least one point of $\mathrm{U}_k(\mathfrak{L})$. Since, by definition, $p\beta q$ implies $(\mathfrak{L}, p) \rightleftarrows (\mathrm{U}_k(\mathfrak{L}), q)$, we can conclude that $\mathfrak{L} \preceq \mathrm{U}_k(\mathfrak{L})$. ◀

To make *all* the temporal successor functions of $\mathfrak{L}$ injective, we apply the untangling operators successively, resulting in $\mathfrak{J} = \mathrm{U}_{n-2}\left(\mathrm{U}_{n-3}(\cdots \mathrm{U}_1(\mathrm{U}_0(\mathfrak{L}))\cdots)\right)$ (where n is the length of $\mathfrak{L}$).

Proposition A.3

Let $\mathfrak{L}$ be a QTG. Then there exists a QTG $\mathfrak{J}$ whose temporal successor functions are all injective, such that:

- $\mathfrak{L} \preceq \mathfrak{J}$;
- $\mathrm{len}(\mathfrak{J}) = \mathrm{len}(\mathfrak{L})$, and $\mathrm{surf}(\mathfrak{J}) \leq \mathrm{len}(\mathfrak{L}) \cdot \mathrm{surf}(\mathfrak{L})$.

Proof. Let $n = \mathrm{len}(\mathfrak{L})$. We define

$$\mathfrak{J} = \mathrm{U}_{n-2}\left(\mathrm{U}_{n-3}(\cdots \mathrm{U}_1(\mathrm{U}_0(\mathfrak{L}))\cdots)\right).$$

Let us check that $\mathfrak{J}$ satisfies the desired properties. In the following, we write, for $k \in [\![-1, n-2]\!]$,

$$\mathfrak{J}_k = \mathrm{U}_k\left(\cdots \mathrm{U}_1(\mathrm{U}_0(\mathfrak{L}))\cdots\right).$$

We can observe that $\mathfrak{J}_{-1} = \mathfrak{L}$, $\mathfrak{J}_{n-2} = \mathfrak{J}$, and for all $k \in [n-1]$, $\mathfrak{J}_k = \mathrm{U}_k(\mathfrak{J}_{k-1})$.

Injectivity Let $k \in [n-1]$: let us show that $\sigma_k^{\mathfrak{J}}$ is injective. First, by definition of the operator U_k, $\sigma_k^{\mathfrak{J}_k} = \sigma_k^{\mathrm{U}_k(\mathfrak{J}_{k-1})}$ is injective. Moreover, an easy induction on ℓ shows that for all $\ell \in [\![k, n-2]\!]$, $\sigma_k^{\mathfrak{J}_\ell} = \sigma_k^{\mathfrak{J}_k}$; in particular, $\sigma_k^{\mathfrak{J}} = \sigma_k^{\mathfrak{J}_{n-2}} = \sigma_k^{\mathfrak{J}_k}$ is injective.

$\mathfrak{L} \preceq \mathfrak{J}$ By Lemma A.2, we have

$$\mathfrak{L} = \mathfrak{J}_{-1} \preceq \mathfrak{J}_0 \preceq \cdots \preceq \mathfrak{J}_{n-3} \preceq \mathfrak{J}_{n-2} = \mathfrak{J}.$$

Since $\preceq$ is a preorder, we can conclude that $\mathfrak{L} \preceq \mathfrak{J}$.

Size control First, since the untangling operators preserve length, we already have $\mathrm{len}(\mathfrak{J}) = \mathrm{len}(\mathfrak{L})$.

Let us now bound the surface of $\mathfrak{J}$. An easy induction on k shows that for all $k \in [\![-1, n-2]\!]$, we have the following:

- $\forall i \in [\![0, k+1]\!],\ |V_i^{\mathfrak{J}_k}| \leq \sum_{j=0}^{i} |V_j^{\mathfrak{L}}|$;
- $\forall i \in [\![k+2, n-1]\!],\ V_i^{\mathfrak{J}_k} = V_i^{\mathfrak{L}}$.

In particular for $k = n-2$, we obtain that for all $i \in [n]$,

$$|V_i^{\mathfrak{J}}| \leq \sum_{j=0}^{i} |V_j^{\mathfrak{L}}| \leq (i+1) \cdot \mathrm{surf}(\mathfrak{L}) \leq n \cdot \mathrm{surf}(\mathfrak{L}) = \mathrm{len}(\mathfrak{L}) \cdot \mathrm{surf}(\mathfrak{L}),$$

so that $\mathrm{surf}(\mathfrak{J}) \leq \mathrm{len}(\mathfrak{L}) \cdot \mathrm{surf}(\mathfrak{L})$. ◀

Remark A.4
In fact, it is not hard to see that we also have $\mathrm{U}_k(\mathfrak{L}) \preceq \mathfrak{L}$, and therefore $\mathfrak{L} \simeq \mathrm{U}_k(\mathfrak{L})$ (where $\mathfrak{A} \simeq \mathfrak{B}$ stands for $\mathfrak{A} \preceq \mathfrak{B}$ and $\mathfrak{B} \preceq \mathfrak{A}$). We therefore also have $\mathfrak{L} \simeq \mathfrak{J}$.

The reason why Theorem 1.7 only says "$\mathfrak{L} \preceq \mathfrak{G}$" is because of the next step in the construction (Section A.2). Indeed, we have $\mathfrak{J} \preceq \mathfrak{G}$ (which gives $\mathfrak{L} \preceq \mathfrak{G}$ and Theorem 1.7), but in general, $\mathfrak{G} \not\preceq \mathfrak{J}$ ($\mathfrak{G}$ is obtained from $\mathfrak{J}$ by adding arbitrary pasts to the nodes in $\mathfrak{J}$ that do not have one; these added nodes in general do not have any equivalent in $\mathfrak{J}$).

A.2. Step 2: building $\mathfrak{G}$

We now turn our attention to building the TG $\mathfrak{G}$ out of $\mathfrak{J}$. This is simply done by adding arbitrary pasts to the nodes in $\mathfrak{J}$ that do not have one.

Proposition A.5

Let $\mathfrak{J}$ be a QTG whose temporal successor functions are all injective. Then there exists a TG $\mathfrak{G}$, such that:

- $\mathfrak{J} \preceq \mathfrak{G}$;
- $\mathrm{len}(\mathfrak{G}) = \mathrm{len}(\mathfrak{J})$, and $\mathrm{surf}(\mathfrak{G}) = \mathrm{surf}(\mathfrak{J})$.

Proof. Let $n = \mathrm{len}(\mathfrak{J})$. Since we can always rename the nodes, we suppose without loss of generality that $V_0^{\mathfrak{J}} \subset \cdots \subset V_{n-1}^{\mathfrak{J}}$ and the $\sigma_i^{\mathfrak{J}}$ are canonical injections (that is, $\sigma_i^{\mathfrak{J}}(v) = v$ for all $v \in V_i^{\mathfrak{J}}$).

Let $\mathfrak{G}$ be the TG of length n defined by:

- $V_0^{\mathfrak{G}} = \cdots = V_{n-1}^{\mathfrak{G}} = V^{\mathfrak{G}} = V_{n-1}^{\mathfrak{J}}$.
- For $i \in [n]$, $E_i^{\mathfrak{G}} = \{(v, w) \in V_i^{\mathfrak{G}} \times V_i^{\mathfrak{G}} \mid (v, w) \in E_i^{\mathfrak{J}}\}$.
- For $v \in V^{\mathfrak{G}}$ and $i \in [n]$,

$$L_i^{\mathfrak{G}}(v) = \begin{cases} L_i^{\mathfrak{J}}(v) & \text{if } v \in V_i^{\mathfrak{J}} \\ \varnothing & \text{else.} \end{cases}$$

- For $i \in [n-1]$, $\sigma_i^{\mathfrak{G}} = \mathrm{id}_{V^{\mathfrak{G}}}$.

The relation $\beta = \{(p, p),\ p \in \mathrm{Pts}(\mathfrak{J})\}$ is a bisimulation between $\mathfrak{J}$ and $\mathfrak{G}$, such that every point of $\mathfrak{J}$ is connected to some point of $\mathfrak{G}$. Since, by definition, $p\beta q$ implies $(\mathfrak{J}, p) \rightleftarrows (\mathfrak{G}, q)$, we deduce that $\mathfrak{J} \preceq \mathfrak{G}$. $\mathrm{len}(\mathfrak{G}) = \mathrm{len}(\mathfrak{J})$ and $\mathrm{surf}(\mathfrak{G}) = \mathrm{surf}(\mathfrak{J})$ are clear. ◀

Theorem 1.7 is then an immediate consequence of Propositions A.3 and A.5.